On Graphs of Incremental Proofs of Sequential Work

Hamza Abusalah[1]

IMDEA Software Institute, Madrid, Spain.
hamza.abusalah@imdea.org

Abstract. In this work, we characterize graphs of *(graph-labeling) incremental proofs of sequential work* (iPoSW). First, we define *incremental* graphs and prove they are necessary for iPoSWs. Relying on space pebbling complexity of incremental graphs, we show that the depth-robust graphs underling the PoSW of Mahmoody et al. are not incremental, and hence, their PoSW cannot be transformed into an iPoSW.
Second, and toward a generic iPoSW construction, we define graphs whose structure is compatible with the incremental sampling technique (Döttling et al.). These are *dynamic* graphs. We observe that the graphs underlying all PoSWs, standalone or incremental, are dynamic. We then generalize current iPoSW schemes by giving a generic construction that transforms any PoSW whose underlying graph is incremental and dynamic into an iPoSW. As a corollary, we get a new iPoSW based on the modified Cohen-Pietrzak graph (Abusalah et al.). When used in constructing blockchain light-client bootstrapping protocols (Abusalah et al.) such an iPoSW, results in the most efficient bootstrappers/provers, in terms of both proof size and space complexity.
Along the way, we show that previous iPoSW definitions allow for trivial solutions. To overcome this, we provide a refined definition that captures the essence of iPoSWs and is satisfied by all known iPoSW constructions.

1 Introduction

A Proof of Sequential Work (PoSW) [15] is a proof system that allows a prover to convince a resource-constrained verifier that it invested a substantial amount of computation sequentially. PoSWs have many applications including time-stamping [15] and blockchain light-client protocols [3, 2, 1].

More formally, a PoSW for a time parameter N is a complete, sound, and succinct proof system (P, V). Completeness guarantees that an honest prover P that makes N scheme-specific computational steps *sequentially* convinces V. For an $\alpha \in [0, 1)$, α-soundness guarantees that a malicious prover $\tilde{\mathsf{P}}$ that makes $\leq \alpha \cdot N$ *sequential* computational steps fails to convince V even if $\tilde{\mathsf{P}}$ has access to massive parallelism. Succinctness demands that the proof size as well as its verification time are small, i.e., $\mathtt{poly}(\log(N), \lambda)$ for a security parameter λ.

An important subclass of PoSWs is Graph-Labeling PoSWs [15, 9, 4, 3] in the parallel random oracle model (PROM). Let $\tau : \{0,1\}^* \to \{0,1\}^\lambda$ be a

random oracle and $\mathcal{G} = (G_n)_{n\in\mathbb{N}}$ a graph family where G_n has a topologically ordered vertex set $V(G_n) := [N]_0 := \{0, 1, \ldots, N\}$ for $N := N(n)$. Then a (graph-labeling) PoSW in the PROM with an underlying graph family $\mathcal{G}$ is an oracle-aided *two-phase interactive* proof system $(\mathsf{P} := \mathsf{P}^{\tau(\cdot)}, \mathsf{V} := \mathsf{V}^{\tau(\cdot)})$. In the first phase, the prover P, upon receiving a high min-entropy bitstring χ from V, salts[1] the RO $\tau(\cdot)$ as $\tau(\chi, \cdot)$ and computes a commitment ϕ to the labels $L := (L(0), \ldots, L(N))$ of the vertex set $[N]_0$. The label of vertex $i \in [N]_0$ is simply defined as $L(i) := \tau(\chi, i, L(\mathsf{parents}(i))$, where $\mathsf{parents}(i)$ are the parents of i given in any fixed order. The second phase is a challenge-response phase in which V samples a challenge set $S_n \subseteq [N]_0$ of size t where t is a scheme parameter. P responds with a proof π_n where π_n in simply a collection of labels, i.e., $\pi_n = L(T_n)$ for a set $T_n \subseteq [N]_0$ that depends on S_n. Finally, V checks π_n and either accepts or rejects. The interactive scheme is then made non-interactive in the PROM using the Fiat-Shamir transform [14].

All known PoSWs [15, 9, 4, 3] are graph-labeling PoSWs in the PROM and they achieve similar asymptotic guarantees. For this exposition, we focus on soundness. They achieve α-soundness for $\alpha \in [0, 1)$, that is, a malicious prover $\tilde{\mathsf{P}}$ that makes a sequence of $\ell \leq \alpha \cdot N$ parallel queries $(Q_1, \ldots, Q_\ell)$, i.e., Q_i is a set of parallel queries, with a total number of queries $q := \sum_{i\in[\ell]} |Q_i|$, makes V accept with probability at most $\alpha^t + \epsilon(q, \lambda)$ where ϵ is a negligible function in λ and a small polynomial in q. (t and λ are security parameters.)

The prover in these schemes [15, 9, 4, 3] has two extreme strategies. On the one extreme, P stores the entire graph labeling L in the first phase, and during the challenge-response phase, it simply compiles its proof π_n from L. (Recall $\pi_n := L(T_n)$ for a succinct subset T_n of $[N]_0$.) This strategy is clearly space inefficient. On the other extreme, P stores only a (succinct) commitment ϕ to L and then *recomputes* the proof labels $L(T_n)$. The first strategy allows P to run in sequential time N, but P must store the entire L, which contains $N+1$ labels. In the second strategy, P stores only a succinct commitment to L, but it must do recomputations that, in the worse case, cost an additional N sequential steps. While this strategy is space-efficient, it creates a soundness slack: whereas the honest prover runs in $2N$ sequential steps, soundness is only guaranteed against αN adversarial sequential steps.

In addition to the above two extreme prover strategies, the PoSW schemes from [9, 4, 3] offer other strategies that admit space-time trade-offs [9, 2]. Concretely, the honest prover can store S labels in the first phase and make V accept while making an additional N/S sequential steps in the second phase. For $\mathrm{S} = \mathrm{T} = \sqrt{N}$, the honest prover makes at most $N + \sqrt{N}$ sequential queries and has $\sqrt{N}$ space complexity.

To resolve the soundness slack in space-efficient PoSWs, Döttling, Lai, and Malavolta [11] introduced and constructed an advanced form of PoSWs called *incremental* PoSWs.

[1] The salting is needed when the RO is instantiated with a hash function.

Incremental PoSWs (iPoSW) [11]. An iPoSW scheme $(\mathsf{P}, \mathsf{V}, \mathsf{Inc})$ is a *non-interactive* PoSW (P, V) with an additional incrementation algorithm Inc. Let N', N'' be integers such that $N = N' + N''$. Then, given a valid PoSW proof π' w.r.t. time parameter N', Inc generates a valid PoSW π w.r.t. N meeting three nontrivial requirements: (a) Inc runs in time essentially N'', (b)[2] P and Inc run in "low" space complexity, i.e., poly-logarithmic in the time parameter, and (c) Inc generates proofs that are identical to those generated by P even if Inc is invoked arbitrarily many times. That is, let $N_1, N_2, \ldots, N_k$ be such that $\sum_{i \in [k]} N_i = N$, then there are two *equivalent* ways of generating the proof π w.r.t. N: (i) directly by running P with parameter N or (ii) by running P w.r.t. parameter N_1 to get π_1 w.r.t. N_1 and then invoking Inc on π_1 and parameter N_2 to get π_2, and on and on until invoking Inc on π_{k-1} and parameter N_k to get π w.r.t. N.

All known iPoSWs [11, 2] are graph-labeling iPoSWs in the PROM and they are transformations of standalone PoSWs. In particular, [11] transforms the standalone PoSW of [9] into an iPoSW and [2] transforms the standalone PoSW of [4, 3] into an iPoSW. These transformations rely on the *incremental sampling* (aka *on-the-fly sampling*) technique [11]: while in a standalone PoSW with underlying graph G_n, P learns the challenge set S_n after P computes the entire labeling L of its vertices $V(G_n) := [N]_0$, in an iPoSW, S_n is sampled *incrementally* while computing the labeling L. On a very high level, the incremental sampling allows P, at various steps during its computation of L, to delete from its memory certain computed labels which it will not need to compile an accepting proof. By the time P computes $L(N)$, P is able to compile an accepting proof from its stored labels. Furthermore, and thanks to the incremental sampling, the total number of stored labels is small, i.e., $\mathtt{polylog}(N)$.

In more detail, the incremental sampling of challenges works as follows. The sampler partitions $V(G_n) := [N]_0$ in topological order into sets $U_1, \ldots, U_k$ of equal size t for some integer t and $t \cdot k = N + 1$. Let $\mathcal{T}$ be the full binary tree with edges emanating from the leaves $U_1, \ldots, U_k$ to the root. Each internal node in $\mathcal{T}$ is a set of size t sampled uniformly at random from its two parent sets. The root of $\mathcal{T}$ is the challenge set S_n for $V(G_n)$. The internal nodes of $\mathcal{T}$ are computed as soon as possible from left to right and bottom up, i.e., sample $U_{1,2}$ from $U_1 \cup U_2$, then $U_{3,4}$ from $U_3 \cup U_4$, then $U_{1,2,3,4}$ from $U_{1,2} \cup U_{3,4}$, then $U_{5,6}$ from $U_5 \cup U_6$ and so on, until sampling the root $S_n := U_{1,\ldots,k}$. For the incremental sampling to be sound, i.e., it gives no advantage to the malicious prover over the original strategy where S_n is sampled after the prover computed and committed to $L(N)$, the sampling randomness, used to sample the intermediate challenge sets along the tree, should not be known in too early, that is, the randomness used to sample $U_{1,2}$ from $U_1 \cup U_2$ is derived using a random oracle invocation on the label $L(2t-1)$, and hence, this ensures that the sampling randomness is only known after one has computed and committed to the entire labeling $L(U_1 \cup U_2)$. The same delayed sampling is ensured across all re-sampled challenge sets. The incremental sampling is sound in the random oracle model [11].

[2] This requirements is missing in previous iPoSW definitions [11, 2], but is satisfied by all previous iPoSW constructions [11, 2].

1.1 Our Contributions.

- Unlike known iPoSW constructions [11, 2], we show that their definitions are in fact vacuous due to their lack of efficiency requirements on the space complexity of honest algorithms. Without stringent space complexity requirements, some standalone PoSWs [9, 4, 3] meet the definition of iPoSWs. Therefore, we give a refined definition of iPoSWs. Our definition captures all known constructions of iPoSWs.
- We define *incremental* graphs and show their necessity to graph-labeling iPoSWs. We define incremental graphs purely combinatorially by using pebbling games on graphs. The definition is simple and clean. The proof of necessity of incremental graphs is simple and relies on a simple lower bound relating (parallel) graph pebbling and graph labeling in the PROM.
- Unlike the PoSWs from [9, 4], which were later transformed into iPoSWs [11, 2], we prove that the first PoSW [15] cannot be transformed into an iPoSW. The PoSW of [15] uses depth-robust graphs [13] with very high depth-robustness. We prove our impossibility in two simple steps. We first observe, due to a result of [5], that graphs with high depth-robustness have high space pebbling complexity. Then we show that the space pebbling complexity of *incremental* graphs is very low. Hence, we conclude that the depth-robustness of the graphs underlying the PoSW [15] makes it impossible to transform such a PoSW into an iPoSW.
- Towards generic iPoSW constructions, we define graphs whose structure is compatible with the incremental sampling technique. These are *dynamic* graphs. We observe that the graphs underlying all PoSW schemes, standalone or incremental, are dynamic.
- We generalize current iPoSWs [11, 2] by giving a generic construction that transforms any PoSW whose underlying graph is incremental and dynamic into an iPoSW. As a testament to the utility of our generic transformation, we get, as a corollary, a new iPoSW based on the modified Cohen-Pietrzak graph $\texttt{CP}^*$ defined in [3]. When used in constructing succinct non-interactive arguments of chain knowledge (SNACK) systems [3, 2, 1], the (incremental) PoSW over the $\texttt{CP}^*$ graph gives the most efficient (incremental) SNACK system. When used in blockchain light-client bootstrapping applications, such iSNACKs allow for space-efficient bootstrappers/provers: such provers would compute/obtain a SNACK proof π_1 w.r.t. an *augmented* blockchain graph on $[N_1]$ vertices, and later, run the incrementation algorithm on π_1 and N_2 to obtain a SNACK proof π for $N = N_1 + N_2$ *without storing* the first N_1 blockchain blocks.

This completes the picture of the PoSW landscape [15, 9, 4, 3]. In particular, standalone PoSWs [9, 4, 3] can be transformed into iPoSWs [11, 2], and the PoSW over depth-robust graphs [15] cannot be made into an iPoSW. Modulo requiring the incremental graphs to be dynamic as well, this establishes the sufficiency and necessity of incremental graphs for iPoSWs. It is important to remark that we don't prove that incremental graphs are sufficient to constructing iPoSWs, but rather, we construct iPoSWs from any PoSW whose graph

is incremental (and dynamic). An interesting open problem we don't address in this work is what graph properties are sufficient for constructing *standalone* (graph-labeling) PoSWs.

1.2 Organization

In Sect. 2, we set up the notation and review the main two tools we need for this work: graph pebbling and graph labeling. In Sect. 3, we revisit the definition of iPoSWs, discuss its inadequacy, and give a refined definition. In Sect. 4, we review some known PoSWs and their graphs and establish some conventions regarding the general syntax of graph-labeling PoSWs. In Sect. 5, we motivate and define *incremental* graphs. In Sect. 6, we prove the necessity of incremental graphs for iPoSWs and show that the PoSW over depth-robust graphs is not incremental. In Sect. 7, we give our generic construction of iPoSWs from PoSWs over dynamic and incremental graphs. Sections 6 and 7 are independent of each other and can be read in the reader's preferred order.

2 Preliminaries

2.1 Notation

Sets and functions. We let $\mathbb{N} := \{0, 1, 2, \ldots\}$, $[N] := \{1, 2, \ldots, N\}$, and $[N]_0 := \{0\} \cup [N]$. We let $\mathtt{poly}(\cdot), \mathtt{polylog}(\cdot)$ and $\mathtt{negl}(\cdot)$ denote the set of all polynomial, poly-logarithmic and negligible functions, respectively.

Graphs. Let $\mathcal{G} = (G_n)_{n\in\mathbb{N}}$ be a directed acyclic graph (DAG) family. We let $V(G_n)$ and $E(G_n)$ denote the vertex and edge sets of G_n. For any DAG G_n, we assume that its indegree is small, i.e., $\mathsf{indegree}(G_n) \in \mathtt{polylog}(N)$, its vertices $V(G_n)$ are topologically ordered into $[N]_0$, it has a unique source $\mathsf{source}(G_n) = 0$ and a unique sink $\mathsf{sink}(G_n) = N$. The $\mathsf{parents}(\cdot)$ operator lists the parents of a vertex in *reverse topological ordering*, i.e., for $v \in G_n$, $\mathsf{parents}(v) := (u_1, \ldots, u_\ell)$ where $\forall i \in [\ell], (u_i, v) \in E_n$ and for $i < j$, u_i precedes u_j topologically. For any DAG G_n and $s \in \mathbb{N}$, we let $H_n := G_n \oplus s$ denote G_n shifted by s, i.e., $V(H_n) = \{s + v : v \in V(G_n)\}$ and $E(H_n) = \{(u + s, v + s) : (u, v) \in E(G_n)\}$.

All graphs in this work are DAGs and we use the words graph and DAG, as well as vertex and node interchangeably.

2.2 The Parallel Random Oracle Model

All graph-labeling PoSW schemes [15, 9, 4, 11, 3, 2] are defined implicitly or explicitly in the Parallel Random Oracle Model (PROM). We define the PROM following [6].

Algorithms in the PROM and Their Complexities. Let $\tau : \{0,1\}^* \to \{0,1\}^\lambda$ be a random oracle and $\mathsf{A}^{\tau(\cdot)}$ a τ-oracle-aided algorithm. $\mathsf{A}^{\tau(\cdot)}$, on an initial state/input, runs in rounds: in the ith round (a) it makes a set of parallel queries Q_i to $\tau(\cdot)$ and receives $\tau(Q_i) := \{\tau(x) : x \in Q_i\}$ and (b) runs an arbitrary computation on its current state and $\tau(Q_i)$ and passes the result as a state to the next round. The *total number of queries* a k-round $\mathsf{A}^{\tau(\cdot)}$ makes is $\sum_{i=1}^{k} |Q_i|$. The *output* of $\mathsf{A}^{\tau(\cdot)}$ is its last state.

We define the *time and space complexities* of $\mathsf{A}^{\tau(\cdot)}$. The *time complexity* of $\mathsf{A}^{\tau(\cdot)}$ is the number of rounds it takes $\mathsf{A}^{\tau(\cdot)}$ to terminate. The *space complexity* of $\mathsf{A}^{\tau(\cdot)}$ is $\max_i\{s_i\}$ where s_i is the space in bits $\mathsf{A}^{\tau(\cdot)}$ uses in its ith round. We will be interested in $\mathsf{A}^{\tau(\cdot)}$ whose internal round computation is either PPT or unbounded.[3]

Graph Labeling. Graph labeling plays a central role in constructing PoSWs. In fact, all known PoSWs [15, 9, 4, 11, 3, 2] are graph-labeling schemes in the PROM, i.e., their computation involves labeling the topologically-ordered vertices of a protocol-specific graph. We first define oracle-based graph labeling in the PROM following [3].

Definition 1 (Graph Labeling [3]). *Let G be a DAG on vertex set $[N]_0$ and $\tau : \{0,1\}^* \to \{0,1\}^\lambda$ an oracle. We define the τ-labeling $L^\tau : [N]_0 \to \{0,1\}^\lambda$ of G recursively as*

$$L^\tau(i) := \begin{cases} \tau(i) & \text{if } \mathsf{parents}(i) = \emptyset \\ \tau\big(i, L^\tau(i_1), \ldots, L^\tau(i_k)\big) & \text{else, i.e., } \mathsf{parents}(i) = (i_1, \ldots, i_k) \end{cases} \tag{1}$$

When τ is clear from context, we simply write L.

For a sequence $S = (s_1, \ldots, s_\ell)$ and a set T, we use the notation $L(S) = (L(s_1), \ldots, L(s_\ell))$ and $L(T) = \{L(t) : t \in T\}$.

2.3 Graph Pebbling

Graph pebbling [12] is a simple combinatorial model of computation in which lower and upper bounds are closely related to those of graph labeling in the PROM. Graph pebbling is extensively studied in the design and analysis of memory-hard functions (MHFs) [6]. In our study of iPoSWs, we use pebble games to abstract the combinatorial properties of PoSW graphs that are necessary for incrementation.

[3] Maybe it would have been more appropriate to call the time complexity of $\mathsf{A}^{\tau(\cdot)}$ the round complexity of $\mathsf{A}^{\tau(\cdot)}$, and then reserve the term time complexity of $\mathsf{A}^{\tau(\cdot)}$ to the time complexity of the round computation of $\mathsf{A}^{\tau(\cdot)}$. However, we opted for our definition in order to unify time complexities of pebble games and graph labeling in the PROM. Additionally, our definition is also compatible with the literature on PoSWs where a prover is said to run in sequential time t if it's time complexity is t according to our convention.

We first define (parallel) pebbling games and their pebbling complexity following [5]. For a DAG G, the (black) pebbling game over G is defined/played in rounds. The goal of the game is to pebble a target subset $S \subseteq V(G)$. Each round $i > 0$ is characterized by its pebbling configuration $C_i \subseteq V(G)$ which denotes the set of vertices currently pebbled, i.e., vertices on which pebbles are placed. For an initial configuration C_0 and $i > 0$, C_i is derived from C_{i-1} according to following rules:

- A vertex $v \in V(G)$ can be pebbled, and hence v is added to C_i, if $\mathsf{parents}(v)$ are pebbled, i.e., $\mathsf{parents}(v) \subseteq C_{i-1}$.
- A pebble can always be removed from any $v \in C_i$.
- A subset $T \subseteq V(G)$ can be pebbled in parallel and its pebbles can also be removed in parallel.

A sequence of configurations $(C_0, \ldots, C_k)$ is legal if it adheres to these rules.

Definition 2 (Pebble Games [5]). *Let G be a DAG, $T \subseteq V(G)$ target set to be pebbled, and C_0 an initial pebbling configuration. A pebbling configuration of G is a subset $C_i \subseteq V(G)$. A* legal *(parallel) pebbling of T is a sequence $C = (C_0, \ldots, C_k)$ of pebbling configurations of G that satisfies the following*

- *Every $v \in T$ is pebbled at some configuration, i.e., $\forall v \in T, \exists i \in [k]_0 : v \in C_i$.*
- *A pebble can be added at a vertex $v \in V(G)$ only when all $\mathsf{parents}(v)$ are pebbled, i.e., $\forall i \in [k] : v \in (C_i \setminus C_{i-1}) \implies \mathsf{parents}(v) \subseteq C_{i-1}$.*

Definition 3 (Pebbling Complexities [5]). *Let G be a DAG, $T \subseteq V(G)$ a target set to be pebbled, and C_0 be an initial pebbling configuration. Let $\mathcal{C}$ be the set of all possible (sequences of) pebbling configurations for T starting in C_0. The* time, space, *and* cumulative space *complexity of a pebbling $C := (C_0, \ldots, C_k) \in \mathcal{C}$ are respectively defined as*

$$\Pi_t(C) := k \ , \qquad \Pi_s(C) := \max_{i \in [k]_0} |C_i| \ , \qquad \Pi_c(C) := \sum_{i \in [k]_0} |C_i| \ .$$

The time, space, and cumulative space pebbling complexity *of G are respectively defined as*

$$\Pi_t(G,T) := \min_{C \in \mathcal{C}} \Pi_t(C) \ , \quad \Pi_s(G,T) := \min_{C \in \mathcal{C}} \Pi_s(C) \ , \quad \Pi_c(G,T) := \min_{C \in \mathcal{C}} \Pi_c(C) \ .$$

2.4 Pebbling and Graph Labeling

We state here known bounds that relate the graph pebbling complexities and graph labeling in the PROM. The time and space complexities in both models are identical modulo a security parameter multiplicative space/pebbles factor. We state this formally in Lemma 1.

Lemma 1 ([12] and/or [6]). *Let G be a DAG, $T \subseteq V(G)$ a target subset and $\tau : \{0,1\}^* \to \{0,1\}^\lambda$ a random oracle.*

1. **Upper bound:** *Let* B *be an algorithm that pebbles* T *from an initial configuration* C_0 *in time and space complexities* $\Pi_t(G,T) = n_t$ *and* $\Pi_s(G,T) = n_s$*, respectively. Then, there exists a PROM algorithm that, given the labeling* $L(C_0)$ *of* C_0*, computes the labeling* $L(T)$ *of* T *in time and space complexities* n_t *and* $n_s \cdot \lambda$*, respectively.*
2. **Lower bound:** *Let* $\mathsf{A}^{\tau(\cdot)}$ *be a PROM algorithm that computes the labeling* $L(T)$ *of* T *in time and space complexities* n_t *and* n_s*, respectively, and it makes in total* q *queries. Then, except with a negligible probability* $(2 + q^2)/2^{\lambda+1}$*,* T *can be pebbled in time and space complexities* n_t *and* n_s/λ*, respectively.*

Remark 1. Lemma 1 of [12] proves the lower bound without considering collisions, which in their scenario don't occur. In their case, the negligible probability is simply $1/2^\lambda$. Collisions under q queries incur an additive factor of $\leq q^2/2^{\lambda+1}$.

3 Incremental PoSWs: A Refined Definition

We give a refined definition of incremental PoSW (iPoSW) in the parallel random oracle model (PROM). Our definition agrees with that of [11, 2] on completeness and soundness but differs on efficiency, which is not defined in [11] and is rather vaguely defined in [2]. We also point out that without our efficiency requirement, the standalone PoSWs [9, 4, 3] are in fact iPoSWs according to the iPoSW definitions of [11, 2], hence, showing the inadequacy of these definitions. Our iPoSW definition is satisfied by all known iPoSW constructions [11, 2] and, as expected, is not satisfied by any of the *standalone* PoSWs [15, 9, 4, 3], hence the appropriateness of our definition.

We refer to Sect. 2.2 for the notation on the time and space complexities of PPT/unbounded algorithms in the PROM.

Definition 4 (Incremental PoSWs). *Let* $(G_n)_{n\in\mathbb{N}}$ *be a DAG family on* $[N]_0$ *vertices with a unique sink* N *where* $N := N(n)$*. A tuple of PPT oracle-aided algorithms* $(\mathsf{P}^{\tau(\cdot)}, \mathsf{Inc}^{\tau(\cdot)}, \mathsf{V}^{\tau(\cdot)} := (\mathsf{V}_0^{\tau(\cdot)}, \mathsf{V}_1^{\tau(\cdot)}))$ *for a random oracle* $\tau : \{0,1\}^* \to \{0,1\}^\lambda$ *is an* incremental proof of sequential work *w.r.t.* τ *if the following hold:*

Completeness: *For every* $\lambda, N \in \mathbb{N}$*, every* $(\chi, N, \mathrm{st}) \leftarrow \mathsf{V}_0^{\tau(\cdot)}(1^\lambda, N)$ *and every*
1. *honestly generated proof* $\pi \leftarrow \mathsf{P}^{\tau(\chi,\cdot)}(1^\lambda, 1^N)$ *and*
2. *honestly incremented proof* $\pi \leftarrow \mathsf{Inc}^{\tau(\chi,\cdot)}(1^\lambda, 1^{N_1}, N_0, \pi_0)$ *for* $N_0, N_1 \in \mathbb{N}$ *s.t.* $N = N_0 + N_1$ *and* π_0 *is* accepting *w.r.t.* N_0*, i.e.,* $\mathsf{V}_1^{\tau(\chi,\cdot)}(\mathrm{st}, N_0, \pi_0) = 1$*,*

then $\mathsf{V}_1^{\tau(\chi,\cdot)}(\mathrm{st}, N, \pi) = 1$*.*

(α, q, ϵ)**-Soundness:** *For every* $\lambda, n \in \mathbb{N}$ *and every unbounded*[4] *adversary* $\tilde{\mathsf{P}}^{\tau(\cdot)}$ *making a total of* q *queries and running in time* $\ell \leq \alpha \cdot N$ *for* $\alpha \in [0,1)$*:*

[4] As defined in the PROM, such an adversary makes a sequence $(Q_1, \ldots, Q_\ell)$ of parallel queries but its computation in each round is unbounded. The only bound we impose is on the total number of queries it makes $q = \sum_{i\in\ell} |Q_i|$.

$$\Pr\left[\begin{array}{c}(\chi, N, \mathrm{st}) \leftarrow \mathsf{V}_0^{\tau(\cdot)}(1^\lambda, N);\\ \pi \leftarrow \tilde{\mathsf{P}}^{\tau(\cdot)}(1^\lambda, 1^N, \chi)\end{array} : \mathsf{V}_1^{\tau(\chi,\cdot)}(\mathrm{st}, N, \pi) = 1\right] \leq \epsilon(\lambda, q).$$

Efficiency: *We require that* P *and* Inc *in the completeness condition meet the following efficiency measures:*

- P *runs in time complexity* N *and space complexity* $\mathtt{poly}(\log(N), \lambda)$ *and its proofs are of* $\mathtt{poly}(\log(N), \lambda)$ *size*
- Inc *for parameter* N_1 *(and input proof* π_0 *for parameter* N_0 *where* $N = N_0 + N_1$*) runs in time complexity* N_1 *and space complexity* $\mathtt{poly}(\log(N), \lambda)$ *and its proofs are of* $\mathtt{poly}(\log(N), \lambda)$ *size.*
- V *runs in time complexity* $\mathtt{polylog}(N)$.

Remark 2. Def. 4 is a two-message protocol. In practice, the first message χ is a high min-entropy string sent from V to P. Then P uses χ to salt a hash function $H(\chi, \cdot)$. In the PROM, no need to send the first message and P simply uses the RO τ defined by the protocol directly, hence the protocol becomes non-interactive. However, we modeled Def. 4 as a two-message, rather than a non-interactive, protocol in order to reflect how such a protocol is used in practice.

Remark 3. A standalone (non-interactive) PoSW scheme is defined similarly except that we don't have the additional $\mathsf{Inc}^{\tau(\cdot)}$ algorithm, i.e., completeness is only required to hold with respect to honestly generated proofs.

Now we discuss the various aspects of our efficiency definition. The more interesting aspects of efficiency that were not defined explicitly, or at all, in precious works [11, 2] are the requirements on the time and space complexities of P and Inc. For completeness, we also discuss succinctness.

Succinctness. The requirement that π is of $\mathtt{poly}(\log(N), \lambda)$ size and that V verifies π in $\mathtt{poly}(\log(N), \lambda)$ time is the same succinctness requirement of [2]. Without succinctness, iPoSWs are trivial to construct in the PROM: let G_n be the line graph, i.e., let $V(G_n) = [N]_0$ where $N = 2^n$ and $E(G_n) = \{(v-1, v) : v \in [N]\}$ and define $(\mathsf{P}, \mathsf{Inc}, \mathsf{V})$ such that P computes the RO-labeling $L := L([N]_0)$ and defines $\pi := L(N)$ as the label of the sink N. Then V recomputes $L(N)$ and accepts if and only if $L(N) = \pi$. Inc works like P. This protocol is complete and sound, but fails to achieve succinctness. So, requiring that π must be short and verifies quickly is essential to avoid such a trivial, and quite useless, construction.

Time Complexity. We start by the time complexity of the honest prover P. Recall P for a time parameter N must run in time (complexity) N, that is, in the PROM, P makes a sequence of parallel queries $(Q_1, \ldots, Q_N)$ and produces a succinct proof π. This very strong requirement is met by all known iPoSWs [11, 2]. Analogous to P, Inc must satisfy a similar strong efficiency requirement: Inc on input a time parameter N_1 and a proof π_0 w.r.t. a time parameter N_0, it outputs the proof π w.r.t. $N_0 + N_1$ in time N_1. (In Appendix A, we elaborate on the time complexity requirement of iPoSWs and the closely related primitive of verifiable delay functions.)

Space Complexity. The space complexity requirement on P and Inc are novel to our definition. We discuss the importance of such a requirement in order to avoid trivial constructions. In particular, without any requirement on the space complexity of P and Inc, the standalone PoSWs of [9, 4, 3] are in fact iPoSWs according to the previous definitions of [11, 2]. Concretely, the prover P in these interactive PoSWs [9, 4, 3] stores the entire graph labeling L in the first phase, and during the challenge-response phase, it simply compiles, without any re-computation, its proof π from L. Such interactive PoSWs are then made non-interactive in the ROM by applying the Fiat-Shamir transform. Therefore, P runs in time complexity N, however its space complexity is N as well.

Without having any space efficiency requirement on P and Inc, one can show that the standalone PoSWs of [9, 4, 3] are in fact iPoSWs according to the iPoSW definitions of [11, 2]. More concretely, and for simplicity, let $N_0 := |V(G_{n-1})|$, $N := |V(G_n)| = 2N_0$, and $N_1 = N_0$. For the PoSWs of [9, 4, 3], Inc can be implemented by using P: on input a time parameter N_1 and a proof π_0 w.r.t. N_0, Inc uses P to label the vertices of G_n by running P *in parallel* to compute the labels of the vertices of (i) G_{n-1} and (ii) the induced sub-graph of G_{n+1} on vertices $V(G_n) \setminus V(G_{n-1})$. This parallel application of P is possible due to the structure of the underlying graphs[5] of the PoSWs of [9, 4, 3]. Inc succeeds in computing $L(V(G_n))$, and hence, producing an accepting π w.r.t. N *in time complexity* N_1; Inc uses P to compile π from $L(V(G_n))$. This shows that the standalone PoSWs of [9, 4, 3] are iPoSWs according to the iPoSW definitions of [11, 2].

Clearly, iPoSWs are meant to achieve more than what standalone PoSWs offer. To fill this gap, we need to impose strong requirements on the space complexity of P and Inc, that is, we require that the space complexities of P and Inc are $\mathtt{poly}(\log(N), \lambda)$. All known iPoSWs [11, 2] meet this natural space complexity requirement and we will argue that none of the standalone PoSWs [15, 9, 4, 3] meet this requirement. Hence, this shows that our definition is appropriate and captures the intuition of existing iPoSW constructions.

To argue that known PoSWs [15, 9, 4, 3] fail to meet our efficiency requirement, observe the prover P in these schemes has other strategies. To begin with, in all these schemes, P has two extreme strategies. The first extreme is the strategy we discussed above where P runs in time complexity N and space complexity N. In the second extreme strategy, P stores only a (succinct) commitment to the graph labels and then *recomputes* the labels needed to compile an accepting proof. In this strategy, while P stores only a succinct commitment to the labeling, it must do recomputations that, in the worse case, cost an additional N

[5] By inspecting these schemes, the only edges $V(G_{n-1})$ sends to $V(G_n) \setminus V(G_{n-1})$ come from a small set $S \subset V(G_{n-1})$ such that $L(S)$ are contained in π_0, and hence, $L(V(G_n) \setminus V(G_{n-1}))$ can be computed given π_0 and no other labels from $L(V(G_{n-1}))$ are needed. In particular, for the `Skiplist` graph (Fig. 2) underlying the PoSWs of [4, 3], $S = \{\mathsf{source}(G_{n-1}), \mathsf{sink}(G_{n-1})\}$, and for the `CP` and $\mathtt{CP}^*$ graphs (Fig. 1) underlying the PoSWs of [9, 3], $S = \{\mathsf{sink}(G_{n-1})\}$.

sequential steps. This means that the time complexity of P is $2N$.[6] This means this extreme strategy fails to meet our definition due to its violation of the time complexity requirement.

Beyond these two extreme strategies, other hybrid prover strategies for the PoSWs of [9, 4, 3] are known [9, 2]. For these schemes, P can produce π in time complexity $N + N'$ and space complexity roughly N/N'. For example, taking $N' = \sqrt{N}$, P runs in time $N + \sqrt{N}$ and space $\sqrt{N}$. These are the best known space-time trade-offs for these schemes, and hence, they don't meet our efficiency requirements.

We close this discussion by re-iterating that our refined definition is met by all known iPoSWs and none of the standalone PoSWs meet our definition.

4 Known PoSWs

In this section, we review the standalone PoSWs and highlight their general syntax. All PoSW schemes [15, 9, 4, 11, 3, 2] in the ROM are constructed in a very similar fashion. Except for the iPoSW schemes of [11, 2], which are defined and constructed non-interactively, the schemes of [15, 9, 4, 3] are constructed in two interactive phases. In the first phase, the prover P computes a labeling L of a graph G_n and sends to the verifier V a commitment ϕ to L. In the second phase, P and V engage in a challenge response phase, in which, V chooses a random set of challenges $S_n \subseteq V(G_n)$, and P answers each challenge $v \in S_n$ with $\pi_n(v) := L(\rho(v))$ where $\rho(v) \subseteq V(G_n)$ is a set of vertices defined by the scheme. We call $\rho(v)$ the *proof vertices* of v. The PoSW proof $\pi_n := \{\pi_n(v) : v \in S_n\}$. The interactive PoSW is made non-interactive using Fiat-Shamir [14].

We formalize the key elements of the syntax of (i)PoSWs below.

Convention 1 ((i)PoSW Proofs) *Let $\mathcal{G} = (G_n)_{n\in\mathbb{N}}$ be a DAG family and $\tau(\cdot)$ a RO. Any PROM (i)PoSW scheme Π, standalone or incremental, over $\mathcal{G}$ samples a* challenge set *$S_n \subseteq V(G_n)$, and for each $v \in S_n$, it defines its* proof vertices *as $\rho(v) \subseteq V(G_n)$, and it finally defines the (i)PoSW* proof *as $\pi_n := \{\pi_n := L(\rho(v)) : v \in S_n\}$ where L is the τ-oracle labeling of G_n (Def. 1). For a set U, we let $\rho(U) := \{\rho(u) : u \in U\}$ and hence we write $\pi_n := L(\rho(S_n))$.*

The $\mathtt{CP}$ *and* $\mathtt{CP}^*$ *Graphs and Their Overlying PoSWs.* We review the graphs underlying the PoSW scheme from [9, 4, 3]. Pictorially, Fig. 1 depicts the Merkle-tree-like PoSW graph underlying the Cohen and Pietrzak PoSW [9], which we call the $\mathtt{CP}$ graph, as well as its modified version, which we call the $\mathtt{CP}^*$ graph. The $\mathtt{CP}^*$ graph was defined and motivated in [3]. (The $\mathtt{CP}$ is also the graph underling the PoSW of [11].)

Definition 5 (The $\mathtt{CP}$ Graph [9]). *We define the $\mathtt{CP}$ graph family $\mathcal{G} := (G_n)_{n\in\mathbb{N}}$ where $V(G_n) = \{0,1\}^{\leq n}$ and $E(G_n) = E'_n \cup E''_n$, where E'_n are the upward edges*

[6] The space complexity of this strategy for the PoSWs [9, 4, 3] is $\mathtt{poly}(\log(N), \lambda)$ while it is roughly $N^2\lambda$ for the PoSW of [15].

of the binary tree of depth n with root ε, the empty string, i.e.,

$$E'_n = \left\{(x\|b, x) : b \in \{0,1\}, x \in \{0,1\}^i, 0 \leq i < n\right\} ,$$

and E''_n contains, for all leaves $v \in \{0,1\}^n$, an edge (u, v) for any u that is a left sibling of a node on the path from v to the root ε in the binary tree $(V(G_n), E'_n)$, i.e.,

$$E''_n = \left\{(u, v) : v \in \{0,1\}^n, v = x\|1\|x', u = x\|0\right\} .$$

Definition 6 (The $\mathtt{CP}^*$ Graph [3]). *We define the $\mathtt{CP}^*$ graph family $\mathcal{G} := (G_n)_{n\in\mathbb{N}}$ where $V(G_n) = \{0,1\}^{\leq n}$ and $E(G_n) = E'_n \cup E''_n$, where E'_n are the upward edges of the binary tree of depth n with root ε, the empty string, i.e.,*

$$E'_n = \left\{(x\|b, x) : b \in \{0,1\}, x \in \{0,1\}^i, 0 \leq i < n\right\} ,$$

and E''_n contains for all nodes v in the tree an edge (u, v) iff u is a left sibling of a node on the path from v to the root ε in the binary tree $(V(G_n), E'_n)$, i.e.,

$$E''_n = \left\{(u, v) : v \in \{0,1\}^i, 0 < i \leq n, v = x\|1\|x', u = x\|0\right\} .$$

Let $V(G_n) = [N]_0$ be given in topological order and let $\mathsf{path}(v)$ denote the shortest path from v to N. Then, by construction [9, 3], the PoSW proof $\pi_n := \{\pi_n(v) := L(\rho(v)) : v \in S_n\}$ where $S_n \subseteq [N]_0$ is a challenge set, $\rho(v) = \{\mathsf{path}(v), \mathsf{parents}(\mathsf{path}(v))\}$ are the proof vertices. For the example $\mathtt{CP}/cpgm$ graph G_4 in Fig. 2, and a challenge $v = 8$, $\pi_4(8)$ consists of $L(\mathsf{path}(8))$ shown in blue and $L(\mathsf{parents}(\mathsf{path}(8)))$ shown in orange. Note asymptotically, $\rho(v) \in O(\log(N))$.

Note while $\mathtt{CP}$ and $\mathtt{CP}^*$ are different as graphs, their overlying PoSW schemes have identical proofs. The only difference between these PoSW schemes is the challenge set $S_n \subseteq V(G_n)$. In the $\mathtt{CP}$ PoSW scheme, S_n is a subset of the leaf nodes, while in the $\mathtt{CP}^*$ PoSW, S_n is a subset of the entire graph vertices.[7]

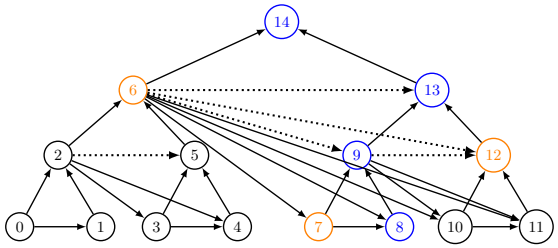

Fig. 1. The $\mathtt{CP}$ and $\mathtt{CP}^*$ graphs. The graph with the dotted and solid edges is the $\mathtt{CP}^*$ graph and the graph with only the solid edges is the $\mathtt{CP}$ graph.

[7] Having a challenge set that comprises the entire graph is the motivation behind defining the $\mathtt{CP}^*$ graph. Such schemes are more SNACK friendly – see [3] for details.

The `Skiplist` *Graph and its Overlying PoSWs.* We review the `Skiplist` graph (Fig. 2) that underlies the PoSW schemes from [4, 3].

Definition 7 (The `Skiplist` Graph [4]). *The* `Skiplist` *DAG family* $\mathcal{G} := (V(G_n), E(G_n))_{n\in\mathbb{N}}$ *is defined as follows.*

$$V(G_n) = [2^n]_0 \text{ and } E(G_n) = \left\{(i,j) \in (V(G_n))^2 : \exists k \geq 0 \text{ s.t. } (j-i) = 2^k \wedge 2^k | i\right\} .$$

Let $N := 2^n$ and $\mathsf{path}(v)$ denote the shortest path from 0 to N passing through v. Then, by construction [4, 2], the PoSW proof $\pi_n := \{\pi_n(v) := L(\rho(v)) : v \in S_n\}$ where $S_n \subseteq [N]_0$ is a challenge set, $\rho(v) = \{\mathsf{path}(v), \mathsf{parents}(\mathsf{path}(v))\}$ are the proof vertices. For the example `Skiplist` graph G_4 in Fig. 2, and a challenge $v = 13$, $\pi_4(13)$ consists of $L(\mathsf{path}(13))$ shown in blue and $L(\mathsf{parents}(\mathsf{path}(13)))$ shown in orange. Note asymptotically, $\rho(v) \in O(\log^2(N))$.

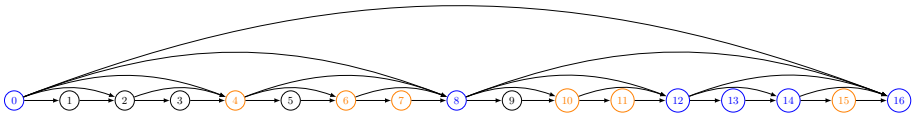

Fig. 2. The `Skiplist` graph G_4.

5 Defining Incremental Graphs

In this section, we define *incremental* graphs. Later we prove that incremental graphs are necessary and sufficient for iPoSWs. That is, we show in Sect. 6.1 that the underlying graph family of any graph-labeling iPoSW must be incremental, and in Sect. 7, we show that any PoSW over an incremental graph family can be transformed into an iPoSW.

To motivate the definition of incremental graphs, recall that by Convention 1, any PoSW scheme Π over a graph family $(G_n)_{n\in\mathbb{N}}$ defines family of challenge sets $(S_n)_{n\in\mathbb{N}}$ where $S_n \subseteq V(G_n)$ and, due to PoSW efficiency requirements, $|S_n| \in \mathtt{polylog}(|V(G_n)|)$. Furthermore, the PoSW proof π_n w.r.t. G_n is defined as $\pi_n := L(\rho(S_n))$ where $\rho(v)$ are the proof vertices of v (Conv. 1) .

For Π to be efficiently incremental, given π_n w.r.t. G_n, it must be the case that the proof $\pi_{n+1} := L(\rho(S_{n+1}))$ w.r.t. G_{n+1} can be computed in time $|V(G_{n+1})| - |V(G_n)|$ and space $\mathtt{poly}(\log(|V(G_{n+1})|), \lambda)$ where λ is a security parameter. (The time and space complexities of PROM algorithms are defined in Sect. 2.2.) In order to isolate the graph properties from the semantics of the PoSW, we capture these requirements combinatorially using the framework of graph pebbling (Sect. 2.4).

Definition 8 (Incremental Graphs). *Let* $\mathcal{G} = (G_n)_{n\in\mathbb{N}}$ *be a DAG family and* $\mathcal{T} = (T_n)_{n\in\mathbb{N}}$ *a set family such that* $T_n \subseteq V(G_n)$ *and* $|T_n| \in \mathtt{polylog}(|V(G_n)|)$.

We say that $\mathcal{G}$ is incremental *if for every $\mathcal{T}$, T_{n+1} can be pebbled given an initial pebbling configuration T_n with time and space complexities*

$$\Pi_t(G_n, T_n) \leq |V(G_{n+1}) \setminus V(G_n)| \quad \textit{and} \quad \Pi_s(G_n, T_n) \in \mathtt{polylog}(|V(G_{n+1})|) \ .$$

To give more insight into the incremental graph definition, we view it in light of known iPoSW graphs. All known iPoSWs [11, 2] have a common feature. Namely, to increment a proof π_n w.r.t. G_n into a proof π_{n+1}, *the incrementation algorithm doesn't compute the label $L(v)$ of any $v \in V(G_{n+1}) \setminus V(G_n)$*. This is so due to two reasons: in these schemes it holds that (a) $\pi_{n+1} \subseteq \pi_n \cup \pi'$ where π' is the PoSW proof w.r.t. the induced sub-graph of G_{n+1} on $V(G_{n+1}) \setminus V(G_n)$ and (b) if $v \notin S_n$, then $v \notin S_{n+1}$, i.e., if a vertex is not a challenge w.r.t. G_n, then it would not be a challenge w.r.t. G_{n+1}.

However, *an iPoSW need not work this way*. That is, both (a) and (b) can be violated, and nevertheless, the iPoSW can maintain its space and time efficiency. To illustrate, assume (b) holds but (a) doesn't. Then the incrementation is efficient as long as π_{n+1} can be computed from π_n time $|V(G_{n+1})| - |V(G_n)|$ and space $\mathtt{poly}(\log(|V(G_{n+1})|), \lambda)$. That is, the proof π_{n+1} need not be a subset of $\pi_n \cup \pi'$. In fact, π' may not even be well defined: the PoSW scheme is only required to define proofs w.r.t. G_n for any $n \in \mathbb{N}$ and need not define proofs w.r.t. the induced sub-graph of G_{n+1} on $V(G_{n+1}) \setminus V(G_n)$. Now assume ($b$) doesn't hold, i.e., $v \notin S_n$ yet $v \in S_{n+1}$, then as long as $L(\rho(S_{n+1}))$ can be computed in time $|V(G_{n+1})| - |V(G_n)|$ and space $\mathtt{poly}(\log(|V(G_{n+1})|), \lambda)$, the incrementation algorithm meets the efficiency requirements. Last, observe that an incrementation algorithm that computes *the labels of the entire graph G_{n+1}* can still be efficient: let G_n be the line graph on $[2^n]_0$ vertices and let $L(0), L(2^n) \in \pi_n$, then the incrementation algorithm can compute $L(1), L(2^n + 1)$ in parallel, then $L(2), L(2^n + 2)$, and so on until it computes $L(2^n), (2^{n+1})$. This takes time 2^n (and not 2^{n+1}) and space $O(\lambda)$.

We close this discussion by remarking that the definition of incremental graphs gives a deeper insight into iPoSWs by decoupling the common features of known iPoSWs from what is really required. We prove the necessity and sufficiency conditions of incremental graphs for iPoSWs in Sect. 6 and 7, respectively.

6 PoSWs over Depth-Robust Graphs are not Incremental

In this section, we prove, in Theorem 1, that the graphs underlying any (graph-labeling) iPoSW must be incremental. Furthermore, in Theorem 2, we prove that the PoSW of [15] based on *depth-robust* graphs [13] is not incremental. This explains why no iPoSW based on the standalone scheme of [15] is known. In contrast, [11] and [2] respectively transform the standalone PoSWs of [9] and [4] into iPoSWs. Not surprisingly, we prove in Sect. 7, Lemma 4, that the graphs underlying the PoSWs from [9, 4] are incremental.

6.1 Incremental Graphs are Necessary for iPoSWs

In this section, we prove in Theorem 1that the graphs underlying any iPoSW must be incremental (as per Def. 8). Given any reasonable definition of incremental graphs, this should not come as a surprise. In a sense, this section is a testament to the appropriateness of our incremental graphs definition.

However, to prove the necessity of incremental graphs for iPoSWs, and avoid trivialities, we put a restriction on the syntax of PoSWs. The restriction is minimal and is met by all known (i)PoSWs [15, 9, 4, 11, 3, 2]. Namely, we assume that the graph-labeling PoSW (P, V) adheres to Convention 1: (P, V) is run over a graph G_n and a proof π_n w.r.t. G_n is simply a collection of labels, i.e., $\pi_n = L(\rho(S_n))$ where $S_n \subseteq V(G_n)$ is a challenge set and V merely checks consistency of $L(\rho(S_n))$.

Without any restriction on its syntax, a PoSW scheme could simply ignore the graph all together and prove sequentiality by other means, say by running a verifiable delay functions (VDF) [7].

Theorem 1 (iPoSWs Have Incremental Graphs). *Let Π be a (graph-labeling) iPoSW with underlying graph family $\mathcal{G}$, then $\mathcal{G}$ is incremental with overwhelming probability.*

Proof. Towards contradiction, assume that $\Pi = (\mathsf{P}, \mathsf{V}, \mathsf{Inc})$ is an iPoSW but $\mathcal{G}$ is not incremental. That is, given an iPoSW proof π_n w.r.t. G_n, which by Convention 1 is a set of labels $\pi_n = L(\rho(S_n))$ for a challenge set $S_n \subseteq V(G_n)$, then, for a challenge set $S_{n+1} \subseteq V(G_{n+1})$ defined by Π, and any valid pebbling for the target set $T_{n+1} = \rho(S_{n+1})$ with time and space pebbling complexities n_t and n_s, respectively, it must be that either $n_t > |V(G_{n+1}) \setminus V(G_n)|$ or $n_s \notin \mathtt{polylog}(|V(G_{n+1})|)$. By the lower bound of Lemma 1, with overwhelming probability $1 - (2 + q^2)/2^{\lambda+1}$, this means that either Inc has PROM time complexity n_t or its PROM space complexity $\lambda \cdot n_s$ is not in $\mathtt{poly}(\log(|V(G_{n+1})|), \lambda)$. In either case, Inc violates the efficiency requirement of an iPoSW (Def. 4), which stipulates that the time complexity of Inc must be $\leq |V(G_{n+1}) \setminus V(G_n)|$ and its space complexity must be in $\mathtt{poly}(\log(|V(G_{n+1})|), \lambda)$. We reach a contradiction. □

6.2 Depth-Robust Graphs Are Not Incremental

The main result of this section is Theorem 2 which shows that PoSWs based on *depth-robust* graphs [13] cannot be incremental. In particular, the PoSW scheme of [15] cannot be made incremental.

We prove Theorem 2 in two steps. First, we observe that depth-robust graphs cannot be pebbled in small space complexity [5]. In fact, their pebbling space complexity is very high. Second, we prove in Lemma 2 that incremental graphs have small (poly-logarithmic) space complexity. Therefore, we conclude that depth-robust graphs are not incremental.

To avoid notational ambiguity, in Lemma 2 below, we let $N_n := N(n)$.

Lemma 2 (Pebbling Complexity of Incremental Graphs). *Let $\mathcal{G} = (G_n)_{n\in\mathbb{N}}$ be an incremental graph family on vertex set $V(G_n) := [N_n]_0$ and $\mathcal{T} = (T_n)_{n\in\mathbb{N}}$ a set family such that $T_n \subseteq [N_n]_0$ and $|T_n| \in \mathtt{polylog}(N_n)$. Then T_n can be pebbled in time and space complexities*

$$\Pi_t(G_n, T_n) \leq N_n \quad \textit{and} \quad \Pi_s(G_n, T_n) \in \mathtt{polylog}(N_n) \ .$$

Proof (Lemma 2). The proof is by induction. Assume we can pebble T_{n-1} in time and space complexities

$$\Pi_t(G_{n-1}, T_{n-1}) \leq N_{n-1} \quad \text{and} \quad \Pi_s(G_{n-1}, T_{n-1}) \in \mathtt{polylog}(N_{n-1}) \ .$$

Then by definition of incremental graphs, given an initial configuration $C_0 = T_{n-1}$, T_n can be pebbled in time and space complexities

$$\Pi_t(G_n, T_n) \leq N_n - N_{n-1} \quad \text{and} \quad \Pi_s(G_n, T_n) \in \mathtt{polylog}(N_n) \ .$$

Therefore, the time and space complexities for pebbling T_n are

$$\begin{aligned}
&\Pi_t(G_n, T_n) \leq N_{n-1} + N_n - N_{n-1} = N_n \\
&\Pi_s(G_n, T_n) \leq \max\{\mathtt{polylog}(N_{n-1}), \mathtt{polylog}(N_n)\} \in \mathtt{polylog}(N_n) \ .
\end{aligned}$$

□

Towards proving that PoSWs based on high depth-robust graphs can't be made incremental, we first review the definition of depth-robust graphs and their *cumulative space pebbling complexity*.

Definition 9 (Depth-Robust Graphs [13]). *Let G be a DAG and $1 \leq e, d \leq |V(G)|$, then G is (e, d)-depth-robust if $\forall S \subset V(G)$ s.t. $|S| \leq e$, the graph $G \setminus S$ contains a path of length at least d.*

Lemma 3 ([5]). *Let G be an (e, d)-depth-robust graph, then $\Pi_c(G) > ed$, where $\Pi_c(G)$ is the cumulative space complexity of G (Def. 3).*

For the design of PoSWs, a depth-robust graph family $(G_n)_{n\in\mathbb{N}}$ on $[N]_0$ vertices with $N = N(n)$ must have high depth robustness, i.e., $(\Theta(N), \Theta(N))$-depth robustness [15]. We observe that graphs with this high depth robustness are not incremental.

Theorem 2. *Let $\mathcal{G} = (G_n)_{n\in\mathbb{N}}$ be a $(\Theta(N), \Theta(N))$-depth-robust graph family where $N = |V(G_n)|$, then $\mathcal{G}$ is not incremental.*

Proof (of Theorem 2). By Lemma 2, if $\mathcal{G}$ is incremental, then any $G_n \in \mathcal{G}$ has time and space pebbling complexities $\Pi_t(G_n) \leq N$ and $\Pi_s(G_n) \in \mathtt{polylog}(N)$, respectively. That is, its *cumulative* space complexity is at most $N \cdot \mathtt{polylog}(N)$. By Lemma 3, the cumulative space complexity of any $(\Theta(N), \Theta(N))$-depth-robust graph is $\Theta(N^2)$. That is, if G_n is incremental, then G_n is not $(\Theta(N), \Theta(N))$-depth-robust. □

We conclude this section by a corollary that answers one of the main questions that initiated this work, namely, can the PoSW scheme of [15] be made incremental? The answer is no, and this explains why previous works [11] and [2] transforming the PoSWs of [9] and [4], respectively, are known, but no iPoSW is known from the first known PoSW of [15].

Corollary 1. *Let Π be the PoSW scheme of [15] which is based on a $(\Theta(N), \Theta(N))$-depth-robust graph family $\mathcal{G} = (G_n)_{n\in\mathbb{N}}$ on $[N]_0$ vertices. Then Π can't be made into an iPoSW with the same graph family $\mathcal{G}$.*

7 Incremental Graphs are Sufficient for iPoSWs

In this section, we show how to transform PoSWs whose underlying graphs are incremental into iPoSWs. (We will additionally require the incremental graphs to be dynamic.) Our transformation is a generalization of known iPoSWs [11, 2]. These schemes themselves are constructed by transforming the standalone PoSWs of [9, 4, 3]. Namely, the iPoSW of [11] transforms the standalone PoSW of [9] based on the `CP` graph (Fig. 1), and the iPoSW scheme of [2] transforms the standalone PoSW of [4, 3] based on the `Skiplist` graph (Fig. 2).

In Sect. 7.1 we highlight the main ideas of the construction; in Sect. 7.2, we give the construction; in Sect. 7.3, we state the main theorem; in Sect. 7.4, we give the security proof.

7.1 An Overview

Let $\mathcal{G} = (G_n)_{n\in\mathbb{N}}$ be an incremental graph family and (P, V) a PoSW over $\mathcal{G}$. Then we show how to construct an incrementation algorithm Inc over the same PoSW. *For the exposition's sake, we will make the simplifying assumption that V challenges are known to P in advance. This assumption is clearly wrong as soundness would not hold.* Later, we will remove this assumption by relying on the incremental sampling of challenges [11, 2].

By Convention 1, every PoSW scheme over a graph family $\mathcal{G}$ defines a family of challenge sets $(S_n)_{n\in\mathbb{N}}$ where $S_n \subseteq V(G_n)$. Furthermore, the PoSW proof π_n w.r.t. G_n is defined as $L(\rho(S_n))$ where $\rho(v)$ are the proof vertices of $v \in S_n$. By our notational conventions (Sect. 2.1), $V(G_n) := [N]_0$ and G_n has a unique source 0 and a unique sink N. We used the shorthand $N := N(n)$ before, and when this shorthand is ambiguous, we would use the shorthand $N_n := N(n)$.

We show how to construct Inc assuming V's challenges S_n are known to P in advance.

$\underline{\mathsf{Inc}^{\tau(\cdot)}}(1^\lambda, 1^{N_n}, N_{n+1}, \pi_n)$: On input a security parameter 1^λ, a time parameter 1^{N_n}, and incrementation parameter N_{n+1}, and a valid proof π_n w.r.t. N_n, $\mathsf{Inc}^{\tau(\cdot)}$ does the following:

1. Parse $\pi_n = L(\rho(S_n))$ where S_n is the challenge set underlying π_n (as chosen by V) and let $T_n = \rho(S_n)$.

2. *Assuming S_{n+1} and hence T_{n+1} is known in advance*, compute $\pi_{n+1} := L(T_{n+1})$ in time $t_1 := N_{n+1} - N_n$ and space $\lambda \cdot \mathtt{polylog}(N_{n+1})$: Incremental graphs guarantee pebbling T_{n+1} in time complexity $\Pi_t(G_{n+1}, T_{n+1}) \leq t_1$ and space complexity $\Pi_s(G_{n+1}, T_{n+1}) \in \mathtt{polylog}(N_{n+1})$. Relying on the upper bound of Lemma 1, pebbles on T_{n+1} can be replaced with labels in the PROM in time and space complexities t_1 and $\lambda \cdot \mathtt{polylog}(N_{n+1})$, respectively.

Inc meets the efficiency requirements of iPoSWs. We now remove the assumption that T_{n+1} is known in advance. T_{n+1} is revealed on the fly as $\mathsf{Inc}^{\tau(\cdot)}$ computes the labels of $V(G_{n+1}) \setminus V(G_n)$. We will show a secure instantiation for this revealing procedure based on what we call *dynamic* incremental graphs and the incremental sampling technique. The structure of these dynamic graphs allows for a simple and sound incremental sampling of T_{n+1}. Resolving the issue of revealing T_{n+1} incrementally in a sound way would complete the picture of the sufficiency of dynamic incremental graphs for iPoSWs.

Note the prover $\mathsf{P}^{\tau(\cdot)}$ itself can run $\mathsf{Inc}^{\tau(\cdot)}$, so it suffices to describe $\mathsf{Inc}^{\tau(\cdot)}$.

Incremental sampling. The only unresolved issue for Inc over incremental graphs is *when* it is safe/sound for Inc to learn S_{n+1}, and hence T_{n+1}. If Inc learns S_{n+1}, or a subset thereof, before it starts computing $L(V(G_{n+1}) \setminus V(G_n))$, then Inc may find a way of computing *some consistent labeling* of T_{n+1} without computing the entire labeling $L(V(G_{n+1}) \setminus V(G_n))$. Such a consistent malicious labeling can be done by fixing arbitrary labels for vertices in T_{n+1} whose parents are not in T_{n+1}, and hence, these fixed labels are not verified by V. However, this cheating strategy might not always work: if T_{n+2} would contain a parent of a node whose label is already fixed, then V rejects. However, this cheating strategy would work for known iPoSWs [11, 2], simply because in these schemes, it holds that for every $v \in V(G_n)$, if $v \notin \rho(S_n)$, then $v \notin \rho(S_{n+1})$.

This shows that learning S_{n+1} too early may endanger soundness of the iPoSW. The incremental sampling technique [11] allows Inc to learn S_{n+1} *incrementally in a sound way*, i.e., without violating soundness of the iPoSW.

In more detail, the incremental sampling of challenges works as follows. The sampler partitions $V(G_n) := [N]_0$ in topological order into sets $U_1, \ldots, U_k$ of equal size t for some integer t and $t \cdot k = N + 1$. Let $\mathcal{T}$ be the full binary tree with edges emanating from the leaves $U_1, \ldots, U_k$ to the root. Each internal node in $\mathcal{T}$ is a set of size t sampled uniformly at random from its two parent sets. The root of $\mathcal{T}$ is the challenge set S_n for $V(G_n)$. The internal nodes of $\mathcal{T}$ are computed as soon as possible from left to right and bottom up, i.e., sample $U_{1,2}$ from $U_1 \cup U_2$, then $U_{3,4}$ from $U_3 \cup U_4$, then $U_{1,2,3,4}$ from $U_{1,2} \cup U_{3,4}$, then $U_{5,6}$ from $U_5 \cup U_6$ and so on, until sampling the root $S_n := U_{1,\ldots,k}$.

For the incremental sampling to be sound, i.e., it gives no advantage to the malicious prover over the original strategy where S_n is sampled after the prover computed and committed to $L(N)$, the sampling randomness, used to sample the intermediate challenge sets along the tree, should not be known in too early, that is, the randomness used to sample $U_{1,2}$ from $U_1 \cup U_2$ is derived using a

random oracle invocation on the label $L(2t-1)$, and hence, this ensures that the sampling randomness is only known after one has computed and committed to the entire labeling $L(U_1 \cup U_2)$. The same delayed sampling is ensured across all re-sampled challenge sets. The incremental sampling is sound in the random oracle model [11].

A minor remark about the incremental sampling is that it can also work with a more relaxed partitioning and a more general ℓ-ary sampling trees. We revisit this point after we formally define dynamic graphs.

Dynamic graphs. We define what we call *dynamic graphs*. Such graphs allow for *efficient* incremental sampling. The efficiency of the sampling comes from the graph structure. We observe that the graphs underlying *all* standalone and incremental PoSWs [15, 9, 4, 11, 3, 2] are dynamic – see Lemma 4.

We recall some necessary notation from Sect. 2.1. For any DAG G_n, we assume that $V(G_n) = [N]_0$ for some integer function $N = N(n)$. For any DAG G_n and $s_n \in \mathbb{N}$, we let $H_n := G_n \oplus s_n$ denote the DAG G_n shifted by s_n, i.e., $V(H_n) = \{v + s_n : v \in V(G_n)\}$ and $E(H_n) = \{(u + s_n, v + s_n) : (u, v) \in E(G_n)\}$.

Informally, a graph family $(G_n)_{n\in\mathbb{N}}$ is *dynamic* if G_{n+1} is constructed from G_n, H_n and an pair of vertex and edge sets (V_n, E_n) in the following manner: $V(G_{n+1}) = V(G_n) \cup V(H_n) \cup V_n$ and $E(G_{n+1}) = E(G_n) \cup E(H_n) \cup E_n$ and s_n, V_n, E_n satisfy some natural conditions that we formalize in Def. 10.

Before formalizing dynamic graphs, let's review two example graphs that are dynamic. The `Skiplist` graph family (Fig. 2) is dynamic: $G_{n+1} \in \mathcal{G}$ is constructed from G_n, H_n where $s_n = |V(G_n)| - 1$ and $V_n = \emptyset$ and $E_n = \{(\mathsf{source}(G_n), \mathsf{sink}(H_n))\}$. (Note due to the shift s_n, it happens that $\mathsf{sink}(G_n) = \mathsf{source}(H_n)$.) Another example of a dynamic graph is the family of full binary trees $(G_n)_{n\in\mathbb{N}}$ on 2^n leaves: G_{n+1} is constructed from G_n and $H_n := G_n \oplus s_n$ where $s_n = |V(G_n)|$ and $V_n = \{r_{n+1} := \mathsf{source}(G_{n+1})\}$ and $E_n = \{(r_{n+1}, \mathsf{source}(G_n)), (r_{n+1}, \mathsf{source}(H_n))\}$.

Definition 10 (Dynamic Graphs). *Let $\mathcal{G} = (G_n, s_n, V_n, E_n)_{n\in\mathbb{N}}$ be a family where G_n is a DAG, s_n an integer s.t. $s_n \geq |V(G_n)| - 1$, and (V_n, E_n) a pair of vertex and edge sets. We say that $\mathcal{G}$ is* dynamic *if G_{n+1} is defined as follows*

1. *$V(G_{n+1}) = V(G_n) \cup V(H_n) \cup V_n$ and V_n satisfies $V_n \cap (V(G_n) \cup V(H_n)) = \emptyset$ and $|V_n| \in \mathtt{polylog}(|V(G_n)|)$.*
2. *$E(G_{n+1}) = E(G_n) \cup E(H_n) \cup E_n$ and E_n satisfies $E_n = \{(u, v) : u, v \in V(G_n) \cup V(H_n) \cup V_n\}$ and $E_n \cap (E(G_n) \cup E(H_n)) = \emptyset$.*

We now justify the choices we made in defining dynamic graphs. First, requiring that $s_n \geq |V(G_n)| - 1$ guarantees that $V(G_n)$ and $V(H_n)$ have *disjoint edges*: when $s_n = |V(G_n)| - 1$, the two graphs share a single vertex, i.e., $V(G_n) \cap V(H_n) = \{\mathsf{sink}(G_n)\} = \{\mathsf{source}(H_n)\}$ and share no edges, i.e., $E(G_n) \cap E(H_n) = \emptyset$. When $s_n > |V(G_n)| - 1$, $V(G_n) \cap V(G_{n+1}) = \emptyset$. One can allow for more general shifts that don't result in disjoint edges, but that would unnecessarily complicate the description of these graphs and the protocols built atop them. Second, restricting the shift s_n such that $E(G_n)$ and $E(H_n)$ are disjoint is without loss of generality as the additional edges E_n is arbitrary.

Dynamic graphs and incremental sampling. The structure of dynamic graphs makes them friendly to incremental sampling. When $s_n \geq |V(G_n)|$ and $V_n = \emptyset$, any graph G_n on $[N]_0$ vertices with $N = t \cdot k - 1$ and $t = |V(G_0)|$ can be partitioned into k subsets $U_1, \ldots, U_k$ each of size t, i.e., for $i \geq 1$, define $U_i = V(G_0 \oplus (i \cdot s_0))$. However, as dynamic graphs allow for overlaps (when $s_n = |V(G_n)| - 1$) and extra vertices (when $V_n \neq \emptyset$), the incremental sampling needs to accommodate these relaxations. In the case when $s_n = |V(G_n)| - 1$, any two consecutive sets U_i, U_{i+1} are allowed to overlap on one vertex – this poses no problem for sampling. This case occurs in the `Skiplist` graph (Fig. 2). In the case when $V_n \neq \emptyset$, we partition G_n into sets $(U_{1,1}, U_{1,2}, V_1), \ldots, (U_{k,1}, U_{k,2}, V_k)$ and build a 3-ary tree such that each internal node defines a challenge set of size t from its 3 parents. This is the case for the `CP` and `CP`* graphs (Fig. 1). For the `CP` and `CP`* graphs, we simply ignore sampling from V_i and sample from $U_{i,1} \cup U_{i,2}$. This is justifiable since by design of the PoSW schemes over these graphs [9, 11, 3], $L(V_i)$ is part of the proof of every challenge chosen from $U_{i,1} \cup U_{i,2}$, and hence, challenging on V_i is redundant. Therefore, and given that, in all known PoSWs, when $V_n \neq \emptyset$, we can ignore sampling from V_n, we stick to sampling from the binary tree and no need to devise a more general incremental sampling as sketched above.

7.2 The Construction

In this section, we give a generic transformation of any PoSW whose underlying graph family is *dynamic and incremental* into an incremental PoSW. Our transformation is a generalization of known iPoSWs [11, 2]. As a testament to the generality of our transform, we get as a corollary an iPoSW from the standalone PoSW of [3] based on the modified `CP`* graph (Fig. 1). This new iPoSW based on the `CP`* graph can be used as is to construct *incremental succinct non-interactive arguments of chain knowledge* (iSNACKs) [3], which can be used to construct blockchain light-client bootstrapping protocols [3, 2, 1]. Note that due to the restricted challenge set of the PoSW [9] and the iPoSW of [11], these schemes are not suitable for (i)SNACKs [3], however, their restricted challenge sets don't hinder their standalone utility and their utility in other applications.

Equipped with the high level overview of Sect. 7.1, we give the formal construction of our transform.

Let $\tau : \{0,1\}^* \to \{0,1\}^\lambda$ be a random oracle and $\Pi' = (\mathsf{P}'^{\tau(\cdot)}, \mathsf{V}'^{\tau(\cdot)})$ a (standalone) PoSW with underlying dynamic and incremental graph family $\mathcal{G} = (G_n, s_n, V_n, E_n)_{n \in \mathbb{N}}$. We transform Π' into an iPoSW $(\mathsf{P}^{\tau(\cdot)}, \mathsf{V}^{\tau(\cdot)}, \mathsf{Inc}^{\tau(\cdot)})$ as follows. The prover and incrementation algorithms work identically, i.e., $\mathsf{P}^{\tau(\cdot)}$ implements $\mathsf{Inc}^{\tau(\cdot)}$ from Sect. 7.1 with the incremental sampling technique. The partitioning of the vertices of G_n into sets $U_1, \ldots U_k$, which serve as leaves in the incremental sampling tree, is not given explicitly in the construction. Rather, $U_1, \ldots, U_k$ are defined implicitly by Algorithm 1. This is in line with previous works [11, 2] and offers efficiency gains, that is, the implicit representation of sets translates into savings in the proof size – $\mathsf{P}^{\tau(\cdot)}/\mathsf{Inc}^{\tau(\cdot)}$ needs not send an

explicit representation of these sets to $\mathsf{V}^{\tau(\cdot)}$. The verifier $\mathsf{V}^{\tau(\cdot)}$ is identical to $\mathsf{V}'^{\tau(\cdot)}$ modulo one technicality: $\mathsf{V}^{\tau(\cdot)}$ needs to additionally verify the correctness of the incremental sampling, i.e., that the incrementally sampled challenged are sampled correctly. To do so, $\mathsf{P}^{\tau(\cdot)}/\mathsf{Inc}^{\tau(\cdot)}$ collects extra indices $\mathcal{I}$ that help $\mathsf{V}^{\tau(\cdot)}$ verify the correctness of the incremental sampling.

In Algorithm 1, where the partitioning of $V(G_n)$ takes place, the concept of re-sampling vertices is introduced. Re-sampling vertices indicate the need to re-sample a challenge set of size t from the parent challenge sets. The label of such vertices define the sampling randomness used in the sampling. More concretely, let $((v,j),(u,j))$ be output by Algorithm 1 when invoked on a vertex w. This means w is a re-sampling vertex and that v and u respectively and implicitly define two challenge sets $S_{v,j}$ and $S_{u,j}$, each of size t. Now $r_{w,j}$ computed as a RO invocation on $(w,j,L(w))$ is the sampling randomness used to sample a challenge set $S_{w,j+1}$ of size t from $S_{v,j} \cup S_{u,j}$.

Algorithm 2 relies on the guarantees of incremental graphs in computing the proofs in time and space complexities within the efficiency requirements of iPoSWs.

On a first reading, the reader is advised to consider the case where $V_n = \emptyset$ (where V_n is the additional vertex set of a dynamic graph) as is the case for the `Skiplist` graph (Fig. 2). In Appendix C, we apply some of the main algorithms of our iPoSW construction to toy examples over the $\mathtt{CP}/\mathtt{CP}^*$ and `Skiplist` graphs.

Parameters. Recall we are given a standalone PoSW $\Pi' = (\mathsf{P}'^{\tau(\cdot)}, \mathsf{V}'^{\tau(\cdot)})$ whose underlying graph family $\mathcal{G} = (G_n, s_n, V_n, E_n)_{n\in\mathbb{N}}$ is dynamic and incremental. We transform Π' into an iPoSW $(\mathsf{P}^{\tau(\cdot)}, \mathsf{V}^{\tau(\cdot)}, \mathsf{Inc}^{\tau(\cdot)})$ as follows.

Let t denote the size of the challenge set of the underlying standalone PoSW Π. For simplicity of exposition, we assume $t = s_0$. Recall, by definition, $s_n \geq |V(G_n)|-1$. If $s_n = |V(G_n)|-1$, and to avoid sampling elements twice, we exclude $\mathsf{source}(G_n)$ and $\mathsf{source}(G_n \oplus s_n)$ from the challenge sets. (When $s_n = |V(G_n)|-1$, this effectively excludes only the 0 challenge as $\mathsf{sink}(G_n)$ is a permissible challenge and $\mathsf{sink}(G_n) = \mathsf{source}(G_n \oplus s_n)$.)

For a random oracle $\tau : \{0,1\}^* \to \{0,1\}^\lambda$ and the verifier's first message χ, we define two oracles τ_ℓ, τ_r as

$$\tau_\ell(\cdot) := \tau(0,\chi,\cdot) \quad \text{and} \quad \tau_r(\cdot) := \tau(1,\chi,\cdot) \tag{2}$$

We use τ_ℓ to label our graphs and τ_r to derive randomness for the incremental sampling.

$\underline{\mathsf{P}^{\tau(\cdot)}(1^\lambda, 1^{N(n)}, \chi)}$:

1. Traverse the graph G_n in topological order, starting from 0. We assume $V(G_n) = [N]_0$ where $N = N(n)$. At every node $w \in [N]_0$ which is traversed, do the following:
   (a) Use $\tau_\ell(\cdot)$ to compute $L(w)$ according to (1). This takes one $\tau_\ell(\cdot)$ query. (Labels that are not explicitly erased are stored in memory.)

(b) Initialize $\mathcal{R} := \emptyset$ and invoke Algorithm 1 on $(w, 0)$ and receive $\mathcal{M}$. (Algorithm 1 determines whether w is a re-sampling point or not.)

(c) If $\mathcal{M} \neq \emptyset$, then w is a re-sampling point and for such w, we re-sample t challenges out of two t-size challenge sets and construct proofs (and indices) for the newly sampled challenges w.r.t. the smallest induced sub-graph containing the original sampling sets. In more detail:

i. Store labels of re-sampling nodes: $\mathcal{R} = \mathcal{R} \cup \{(w, L(w))\}$

ii. Traverse $\mathcal{M}$ in *increasing* order of j where $((v, j), (u, j)) \in \mathcal{M}$.

A. Define the set of additional (shifted) vertices V'_j as

$$V'_j := \begin{cases} \emptyset & \text{if } V_j = \emptyset \\ \{v+1, \ldots, v+|V_j|\} & \text{if } V_j \neq \emptyset \end{cases}$$

(Note each dynamic graph (G_j, s_j, V_j, E_j) defines such V_j.)

B. If $j = 0$, define for $z \in \{u, v\}$

$$\mathcal{L}_{z,0} := L((z, z-1, \ldots, z-t+1)) \tag{3}$$
$$\mathcal{I}_{z,0}[k] := k \quad \forall k \in [t] \tag{4}$$

($\mathcal{L}$ contains PoSW proofs and $\mathcal{I}$ contains indices related the incremental sampling and would be needed for V (in Alg. 3.) to verify the veracity of the sampling.)

C. Re-sample challenges: use randomness $r_{w,j} := \tau_r(w, j, L(w))$ to sample a random subset $S_{w,j+1}$ of size t from the set $[2t]$. (Note while $S_{w,j+1} \subset [2t]$, it defines (as in Alg. 2) a set of *actual* challenges, i.e., graph vertices that V checks.)

D. Compile proofs (and indices) for the new challenges by running Alg. 2 on $(\mathcal{L}_{v,j}, \mathcal{L}_{u,j}, \mathcal{I}_{v,j}, \mathcal{I}_{u,j}, S_{w,j+1}, L(V'_j))$ to obtain $\mathcal{L}_{w,j+1}$ and $\mathcal{I}_{w,j+1}$.

E. Store $\mathcal{L}_{w,j+1}$ and $\mathcal{I}_{w,j+1}$ in memory and erase $\mathcal{L}_{u,j}$, $\mathcal{L}_{v,j}$, $\mathcal{I}_{u,j}$, $\mathcal{I}_{v,j}, L(V_i)$ from memory.

2. Terminate and output $\pi := (\mathcal{L}_{w,j+1}, \mathcal{I}_{w,j+1}, \mathcal{R}, j+1)$.

$\underline{\mathsf{Inc}^{\tau(\cdot)}(1^\lambda, 1^{N_1}, N_0, \pi, \chi)}$:

1. Parse π as $(\mathcal{L}, \mathcal{I}, \mathcal{R}, d)$.
2. Compute $N = N_0 + N_1$ and check that $N = |V(G_n)|$ for some $n \in \mathbb{N}$.
3. Execute the algorithm $\mathsf{P}^{\tau(\cdot)}(1^\lambda, 1^{N_1}, \chi)$ starting from Step 1a with a slight change: traverse the graph starting from $N_0 + 1$.

$\underline{\mathsf{V}^{\tau(\cdot)}(1^\lambda, N, \pi, \chi)}$:

1. Parse π as $(\mathcal{L}, \mathcal{I}, \mathcal{R}, d)$ and let $\mathcal{R}[|\mathcal{R}|] = (w, L(w))$ be the last re-sampling point.
2. For all $j \in [t]$, run Algorithm 3 on input $(\mathcal{L}[j], \mathcal{I}[j], \mathcal{R}, d, 0, w)$ and let b_j be its output bit.
3. Return $\bigwedge_{j=1}^{t} b_j$.

Algorithm 1. This algorithm determines whether a node v is a re-sampling node, and if so, returns the set of all pairs of nodes $\mathcal{M}$ from which to re-sample challenges: if $((v,j),(u,j)) \in \mathcal{M}$, then we re-sample the challenges associated with (v,j) and (u,j). If $\mathcal{M} = \emptyset$, then v is not a re-sampling point. For example, for $v = 16$ and $t = |V(G_0)| - 1 = 2$ in Fig. 2, $\mathcal{M} = \{((16,2),(8,2)),((16,1),(12,1)),((16,0)(14,0))\}$.

Input: (v, s)
$\mathcal{M} = \emptyset$
if $v \leq |V(G_0)| - 1$ **then**
  return $\mathcal{M}$
end if
▷ Recall G_{j+1} is composed from $G_j, G_j \oplus s_j, V_j, E_j$; see Def. 10
if $v = |V(G_j)| - 1$ for some integer $j \in [i]$ **then**
  $u := v - s_{j-1}$
  add $((v + s - |V_{j-1}|, j-1),(u + s - |V_{j-1}|, j-1))$ to $\mathcal{M}$
end if
$v = v - s_{j-1}, \quad s = s + s_{j-1}$ ▷ Note that $|V(G_{j-1})| < v + 1 \leq |V(G_j)|$
Recurse on (v, s)

7.3 The Theorem Statement and Instantiations

Before we state and prove our main theorem, we state as a convention a simple observation about the soundness proof of all known PoSWs [15, 9, 4, 11, 3, 2], whether standalone or incremental.

Convention 2 *When referring to (α, ϵ)-sound graph-labeling PoSWs, we assume their security analysis relies on the sequentiality of random oracles and that their soundness error is of the form $\epsilon = \alpha^t + \mathsf{negl}(\lambda)$ where t is the number of challenges and λ is the bit length of the random oracle's output.*

In fact, all PoSWs rely on the sequentiality of random oracles, captured by Lemma 5, in a simple manner.[8] In more detail, in the (α, ϵ)-soundness security proof, from any malicious prover $\tilde{\mathsf{P}}$ and its queries, a *τ-sequence* (Def. 12) of length $\alpha \cdot N$ is extracted. Such a sequence is unique unless $\tilde{\mathsf{P}}$ finds collisions in the RO $\tau(\cdot)$ (which defines the labeling L) or guesses its output. So, except with a negligibly small probability, due to the sequentiality of τ-sequences, this means that if V accepts after submitting t challenges, then $\tilde{\mathsf{P}}$ must have made $\geq \alpha \cdot N$ sequential queries except with $\alpha^t + \mathsf{negl}(\lambda)$ probability.

With this convention in mind, we state our main theorem.

Theorem 3 (From PoSWs to iPoSWs). *Let λ and t be security parameters, $(\mathsf{P}', \mathsf{V}')$ be an $(\alpha, \alpha^t + \mathsf{negl}'(\lambda))$-sound (graph-labeling) PoSW with an incremental and dynamic graph family $\mathcal{G} = (G_n, s_n, V_n, E_n)_{n \in \mathbb{N}}$ (Def. 10 and Def. 8) on $[N]_0$ vertices where $N = N(n)$. The construction $(\mathsf{P}^{\tau(\cdot)}, \mathsf{V}^{\tau(\cdot)}, \mathsf{Inc}^{\tau(\cdot)})$ from Sect. 7.2*

[8] In some works such as [9], such lemma is stated more generally, and in some other works such as [11], it's implicit in the security analysis.

Algorithm 2. This algorithm given sub-proofs $\mathcal{L}_{v,j}, \mathcal{L}_{u,j}$ and indices $\mathcal{I}_{u,j}, \mathcal{I}_{u,j}$ associated with $((u,j)(v,j)) \in \mathcal{M}$, outputs a sub-proof and indices $\mathcal{L}_{w,j+1}, \mathcal{I}_{w,j+1}$ for the corresponding re-sampling point w. The algorithm also gets the re-sampled set of indices S and some auxiliary labels $L(V'_j)$

▷ For a graph G, we use $L(G)$ to denote its labels $L(V(G))$.

Input: $(\mathcal{L}_{v,j}, \mathcal{L}_{u,j}, \mathcal{I}_{u,j}, \mathcal{I}_{u,j}, S, L(V'_j))$

Let $K_{z,j}$ be the induced sub-graph on vertices $\{z, z-1, \ldots, z-|V(G_j)|-1\}$. By design, we have

$$L(K_{w,j+1}) = L(K_{u,j}) \cup L(K_{v,j}) \cup L(V'_j) \quad \text{and} \quad K_{v,j} = K_{u,j} \oplus s_j$$

Initialize $\mathcal{L}_{w,j+1}, \mathcal{I}_{w,j+1} = \emptyset$

for $k \in [t]$ **do**

$w_k := u + S[k]$ ▷ w_k is the actual challenge node.

▷ We assumed here S is given in an increasing and that $S[i]$ is its ith element.

if $j = 0$ **then** ▷ Construct base proofs using the standalone PoSW prover P'.

Compile $L(K_{w,1})$ from $\mathcal{L}_{u,0}, \mathcal{L}_{v,0}, L(V_0)$ by noting that

$$L(K_{w,1}) = L(K_{u,0}) \cup L(K_{v,0}) \cup L(V_0)$$
$$L(K_{u,0}) = \mathcal{L}_{u,0}$$
$$L(K_{v,0}) = \mathcal{L}_{v,0}$$

Run P' on $L(K_{w,1})$ to obtain the PoSW proof $\pi_1(w_k)$ w.r.t. $K_{w,1}$.

▷ $\pi_1(w_k)$ is the proof for challenge w_k w.r.t. $K_{w,1}$.

end if

if $j > 0$ **then** ▷ Construct proofs given the incremental guarantees of graphs.

Compute $\pi_{j+1}(w_k)$ w.r.t. $K_{w,j+1}$ based on the guarantees of incremental graphs (Def. 8): compute the labels of $\rho(w_k)$ and set $\pi_{j+1}(w_k) := L(\rho(w_k))$. By Convention 1, $\pi_{j+1}(w_k)$ is a valid PoSW for challenge w_k. Computing $\pi_{j+1}(w_k)$ is done given the proof $\pi_j(w_k)$ w.r.t. G_j in time $t_{j+1} := |V(K_{w,j+1})| - |V(K_{u,j})|$ and space $\lambda \cdot \mathtt{polylog}(t_{j+1})$.

end if

$\mathcal{L}_{w,j+1}[k] = \pi_{j+1}(w_k)$

Write $S[k] = a \cdot t + b$ with the smallest $a \in \{0,1\}$ and $b \in [t]$

Compute sampling indices

$$\mathcal{I}_{w,j+1}[k] = \begin{cases} (\mathcal{I}_{u,j}[b], k) & \text{if } a = 0 \\ (\mathcal{I}_{v,j}[b], k) & \text{if } a = 1 \end{cases}$$

▷ We interpret $\mathcal{I}$ as a sequence $(i_1, \ldots, i_\ell)$ and let $(\mathcal{I}, k) := (i_1, \ldots, i_\ell, k)$

end for

return $\mathcal{L}_{w,j+1}, \mathcal{I}_{w,j+1}$

Algorithm 3. This algorithm on input a proof π and some auxiliary input $(\mathsf{ind}, \mathcal{R}, \mathsf{d}, \mathsf{s}, \mathsf{e})$ related to the re-sampling process, it (a) retrieves the challenge consistent with the sampling process and (b) verifies using V' the proof π for that challenge and outputs the verdict bit.

Input: $(\pi, \mathsf{ind}, \mathcal{R}, \mathsf{d}, \mathsf{s}, \mathsf{e})$
if $\mathsf{d} = 0$ **then**
  if $t = |V(G_0)|$ **then**
    $v := \mathsf{s} + \mathsf{ind}[1] - 1$ ▷ v is the challenge node.
  end if
  if $t = |V(G_0)| - 1$ **then**
    $v := \mathsf{s} + \mathsf{ind}[1]$ ▷ Challenges start from 1 rather than 0.
  end if
  Return $\mathsf{V}'(v, \pi)$ ▷ V' verifies the standalone PoSW proof for v.
end if
if $\mathsf{d} > 0$ **then**
  ▷ Recall G_{i+1} is composed from $G_i, G_i \oplus s_i, V_i, E_i$; see Def. 10.
  Parse ind as $\mathsf{ind} := (i_0, \ldots, i_{\mathsf{d}}) \in [t]^{\mathsf{d}+1}$
  $r = \tau_r(\mathsf{e}, \mathsf{d} - 1, L(\mathsf{e}))$ where $L(\mathsf{e})$ is retrieved form $\mathcal{R}$
  ▷ Recall $\mathcal{R} = \{(v, L(v)) : v \text{ is a re-sampling point}\}$.
  Use r to choose a random subset S of size t from $[2t]$
  Write $S[i_{\mathsf{d}}]$ as $S[i_{\mathsf{d}}] = a \cdot t + b$ with smallest $a \in \{0, 1\}$ and $b \in [t]$
  $\mathsf{m} := \mathsf{e} - |V_{\mathsf{d}-1}| - (1 - a) \cdot s_{\mathsf{d}-1}$
  ▷ m is the previous re-sampling node.
  if $a = 0$ **then**
    Recurse on $(\pi, (i_0, \ldots, i_{\mathsf{d}-1}), \mathcal{R}, \mathsf{d} - 1, \mathsf{s}, \mathsf{m})$
  end if
  if $a = 1$ **then**
    Recurse on $(\pi, (i_0, \ldots, i_{\mathsf{d}-1}), \mathcal{R}, \mathsf{d} - 1, \mathsf{s} + m_{\mathsf{d}-1}, \mathsf{m})$
  end if
end if

is an (α, ϵ)*-sound iPoSW with*

$$\epsilon = q \cdot e^{-2t\cdot\left(\frac{1-\alpha}{\log(N)}\right)^2} + \mathsf{negl}(\lambda) \quad \textit{where} \quad \mathsf{negl}(\lambda) = \frac{4+3q^2-q}{2^{\lambda+1}} \ ,$$

where q *upper-bounds the total number of queries to the* random oracle *and is bounded by a sub-exponential in* λ*. (Think of* $q = 2^{40}$ *and* $\lambda = 256$*.)*

Security. In the security proof, we rely on the sequentiality of random oracles of the underlying standalone PoSW in a black-box manner, but we reprove the negl term. We do so, for the sake of giving a complete construction and proof of security without overwhelming the construction with unnecessary abstractions.

Modulo the essentially identical negligible terms, Thm. 3 transforms an α^t-sound standalone PoSW into a $(q \cdot e^{-2t\cdot((1-\alpha)/\log(N))^2})$-sound iPoSW. Thee soundness loss, manifested by the quadratic dependence on $\log(N)$, persists in all other iPoSWs [11, 2] and is due to the use of a Hoeffding bound in the security analysis of the underlying incremental sampling technique. We don't know how to overcome this soundness loss.

Instantiations. Lemma 4 below proves that the `CP` graph underlying the PoSW from [9, 11], the `CP`* graph underlying the PoSW from [3], and the `Skiplist` graph underlying the PoSW from [4, 3, 2] are dynamic and incremental. (The `CP`, `CP`*, and `Skiplist` graphs are defined in Sect. 4 and depicted in Fig. 1 and 2.) Therefore, Theorem 3, via Corollary 2 below, gives a unified construction for all these graphs. In particular, the iPoSWs of [11, 2] based on the `CP` and `Skiplist` graphs follow, as special cases, from Theorem 3. While these iPoSWs over the `CP` and `Skiplist` graphs are not new, the iPoSW over the `CP`* graph is new. This scheme is very similar to the iPoSW of [11]. The only difference is that while the standalone PoSW [9] and its incremental counterpart [11] based on the `CP` graph restrict the verifier challenges to the leaves of the Merkle-tree-like `CP` graph, the standalone PoSW based on the `CP`* graph [3] and its incremental counterpart (Corollary 2) allow the challenges to be sampled from *all the vertices* of the `CP`* graph. Allowing sampling challenges from all possible graph vertices leads to light-client blockchain bootstrapping applications via (i)SNACKs [3].

Corollary 2. *Let* Π' *be any standalone PoSW from [9, 4, 3] with underlying graph family* $\mathcal{G}$*. Then* Π *guaranteed by Theorem 3 is an iPoSW over* $\mathcal{G}$*.*

Proof. It suffices to show that $\mathcal{G}$ is incremental and dynamic. This we show in Lemma 4. The corollary follows from Theorem 3. □

Lemma 4. *The* `CP` *[9],* `CP`* *[3], and* `Skiplist` *[4, 3] graphs are dynamic (Def. 10) and incremental (Def. 8).*

The proof of Lemma 4 is simple and is given in Appendix B.

7.4 The Security Proof

In this section, we prove that the construction from Sect. 7.2 is an iPoSW.

Completeness and Efficiency. Completeness and efficiency follow straightforwardly from the guarantees of incremental and dynamic graphs. Concretely, from the guarantees of dynamic graphs, G_n is constructed from $(G_{n-1}, s_{n-1}, V_{n-1}, E_{n-1})$, and for simplicity, we assume $V_{n-1} = \emptyset$ and $s_{n-1} = |V(G_{n-1})|$. From the guarantees of incremental graphs, given π_{n-1} w.r.t. G_{n-1} and a challenge set S_n, $\rho(S_n)$ can be pebbled in time $t_n = |V(G_n) \setminus V(G_{n-1})| = |V(G_{n-1})|$ and space $\mathtt{polylog}(|V(G_n)|)$. Now S_n is computed incrementally: given the dynamic structure of G_n, the incremental sampling partitions $V(G_n) = [N]_0$ into sets $U_1, \ldots, U_k$ where $U_i = V(G_0 \oplus (i \cdot s_0))$ and builds the full binary tree $\mathcal{T}$ with $U_1, \ldots, U_k$ as its leaves. Each internal node in $\mathcal{T}$ is a set of size t sampled uniformly at random from its two parent sets. The root of $\mathcal{T}$ is the challenge set S_n w.r.t. G_n. The prover P, follows this tree structure. It follows then that G_n can be pebbled in time complexity t_n and space complexity $\mathtt{polylog}(|V(G_n)|)$, Hence, by Lemma 1, G_n can be labeled in time t_n and space $\lambda \cdot \mathtt{polylog}(|V(G_n)|)$. Now the tree structure affects the space and parallelism of P by a multiplicative $\log(N)$ factor, i.e., the depth of the sampling tree. As $\log(N) \cdot \mathtt{poly}(\log(N), \lambda) \in \mathtt{poly}(\log(N), \lambda)$, this factor has no asymptotic effect on the space complexity of P. Modulo the index set $\mathcal{I}$ and the resampling set $\mathcal{R}$, the prover P generates the same proofs P' would generate. $\mathcal{I}$ contains $\log(N)$ indices, each of size $\log(t)$ bits. $\mathcal{R}$ contains $\log(N)$ pairs $(v, L(v))$, each of size $\log(N) + \lambda$ bits. Therefore, as P' generates (by definition) proofs of size $\mathtt{poly}(\log(N), \lambda)$, the proof generated by P (and by design by Inc) are of $\mathtt{poly}(\log(N), \lambda)$ size. As for the verification time of V, note that V, in addition to running V', must verify the veracity of the incremental sampling (see Alg. 3). However, this incurs a $\log(N)$ factor in the running time of V, and as V' runs in $\mathtt{poly}(\log(N), \lambda)$ time, so does V. This concludes the completeness and efficiency aspects of our iPoSW.

Soundness. We start off by recalling basic notions and lemmas. All known graph-labeling (i)PoSW schemes are in the random oracle model. Their proofs of security rely on a lemma about the sequentiality of random oracles. To state such lemma, we first need the concept of a consistent string.

Definition 11 (Consistent Strings [3]). *For an oracle $\tau : \{0,1\}^* \to \{0,1\}^\lambda$ and a DAG $G = ([N]_0, E)$, define $G^+ = ([N+1]_0, E^+)$ with $E^+ = E \cup \{(N, N+1)\}$. Furthermore, $\forall i \in [N+1]_0$, let p_i be the number of parents of i in G^+ and $y_i := (i, y_i[1], \ldots, y_i[p_i]) \in ([N]_0 \times (\{0,1\}^\lambda)^{p_i})$. We say y_i is* consistent *with $y_{i'}$ w.r.t. G, and denote it by $y_i \prec y_{i'}$ if $(i, i') \in E^+$ and if i is the j-th parent of i' in G^+ (in reverse topological order), then the j-th block in the decomposition of $y_{i'}$ is equal to $\tau(y_i)$, i.e., $y_{i'}[j] = \tau(y_i)$.*

In a PoSW over a graph over $[N]_0$ vertices, an honest prover P produces a sequence $Q = (Q_0, \ldots, Q_N)$ of queries with $Q_i := \{(i, L(\mathsf{parents}(i)))\}$ for every $i \in [N]_0$. To capture the sequential work of malicious provers, the notion of τ-sequence is defined.

Definition 12 (τ-sequences [3]). *Let $G = ([N]_0, E)$ be a DAG and $\tau : \{0,1\}^* \to \{0,1\}^\lambda$ an oracle. A sequence of strings $s := (y_{i_1}, \ldots, y_{i_{\ell+1}})$ with $y_{i_{\ell+1}} \in \{0,1\}^\lambda$ and $y_{i_j} \in ([N]_0 \times (\{0,1\}^\lambda)^{|\mathsf{parents}(i_j)|})$ is called a* τ-sequence of length ℓ *if $\forall j \in [\ell], y_{i_j} \prec y_{i_{j+1}}$ w.r.t. G.($\prec$ is as in Def. 11.)*

Note that the sequence Q of queries of an honest prover can be made into a τ-sequence $(y_0, \ldots, y_N, y_{N+1})$ of length N where $\forall i \in [N]_0, y_i := (i, L(\mathsf{parents}(i)))$ and $y_{N+1} := \tau(y_N)$.

Lemma 5 shows that, except with negligible probability, no malicious prover $\tilde{\mathsf{P}}$ which makes a sequence of parallel queries Q of length ℓ can produce a τ-sequence of length $> \ell$ when τ is a random oracle.

Lemma 5 (Sequentiality of Random Oracles [3]). *Let G be a DAG on $[N]_0$ vertices and $\tau : \{0,1\}^* \to \{0,1\}^\lambda$ a random oracle. Let $\tilde{\mathsf{P}}^{\tau(\cdot)}$ be a malicious prover that makes a sequence of (parallel) queries $(Q_0, \ldots, Q_\ell)$ to $\tau(\cdot)$ for $\ell = \alpha N$ and $\alpha < 1$ and makes q oracle queries in total, then*

$$\Pr\left[s \leftarrow \tilde{\mathsf{P}}^{\tau(\cdot)} : \ s \text{ is a } \tau\text{-sequence of length} > \ell\right] \leq \frac{1+q^2}{2^\lambda} \ .$$

where the probability is over the choice of τ and the randomness of $\tilde{\mathsf{P}}^{\tau(\cdot)}$.

Soundness. Now we turn into proving soundness. The hybrid games and their proofs are almost identical to those of [2]. This is not surprising as our construction is an abstracted version of the constructions from [11, 2]. Recall that oracle access to τ implies oracle access to τ_ℓ, τ_r defined in (2) and that τ is modeled as a random oracle.

Recall G_n is a dynamic and incremental graph on $[N]_0$ vertices.

$\mathsf{Exp}^{\tau(\cdot)}_{\tilde{\mathsf{P}},0}(1^\lambda, 1^N) \in \{0,1\}$*:* This is the original experiment.

$\mathsf{Exp}^{\tau(\cdot)}_{\tilde{\mathsf{P}},1}(1^\lambda, 1^N) \in \{0,1\}$*:* This is similar to $\mathsf{Exp}^{\tau(\cdot)}_{\tilde{\mathsf{P}},0}$ except that we reject if the proof is not *extractable* from $\tilde{\mathsf{P}}$'s queries.

Towards defining extractable proofs, we first define the notion of query graphs. Sample $\chi \leftarrow \{0,1\}^\lambda$, run $\pi \leftarrow \tilde{\mathsf{P}}^{\tau(\cdot)}$, and observe its queries $K := \{(x, y) : y = \tau_\ell(x)\}$ and τ_ℓ is as in (2). Use K to build its query graph $\mathsf{QG} = (V, E)$: let $V, E = \emptyset$, then for every $v_i := (x_i, y_i), v_j := (x_j, y_j) \in K$, add v_i, v_j to V and (v_i, v_j) to E if and only if $x_i \prec x_j$ w.r.t. G_n and the operator $\prec$ as in Def. 11.

By Convention 1, we have that $\pi = L(\rho(S_n))$ where S_n is V's challenge set – which, in our case, is incrementally sampled and is implicitly defined by the protocol. We say π is *extractable* from QG if for every label $z \in \pi$, either z is a query output, i.e., $((v, x'), z) \in \mathsf{QG}$ or z is a part of a query input, i.e., $((v, \cdots, z, \cdots), y) \in \mathsf{QG}$.

We define a series of experiments, the first of which coincides with $\mathsf{Exp}^{\tau(\cdot)}_{\tilde{\mathsf{P}},1}$ and the probability of success in the last reduces to the security guarantees of the underlying standalone PoSW.

Recall that in Item 1b, we invoke Algorithm 1 on $(w, 0)$ to get the set $\mathcal{M}$ of re-sampling points associated with w, and in Item 1(c)iiC, we re-sample a set of t indices $S_{w,i}$ from two sets of indices, call them $U_{u,i-1}$ and $U_{v,i-1}$ and they are respectively associated with $(u, i-1)$ and $(v, i-1)$ where $((u, i-1), (v, i-1)) \in \mathcal{M}$. Define $S'_{w,i}$ to be the set of challenge nodes corresponding to the indices in $S_{w,i}$ and let $\beta := (w, i)$ and $\mathsf{chal}(\beta) := S'_{w,i}$.

Now let $\mathcal{B}$ be the ordered (in increasing order) set of all re-sampling points during the execution of the protocol. Furthermore, let $\beta_s = (w_s, i_s)$ be the last element in $\mathcal{B}$.

For each incremental re-sampling $\beta = (w, i) \in \mathcal{B}$, we define a bad event bad_β and a corresponding experiment that outputs 1 if and only if $\mathsf{bad}_\beta = 1$. The event $\mathsf{bad}_\beta = 1$, informally speaking, indicates that the incremental sampling favors $\tilde{\mathsf{P}}$. Formally, we define functions $\delta, \gamma, \eta : \mathcal{B} \to [0, 1]$, event bad_β, and $\mathsf{neighbor} : \mathcal{B} \to \mathcal{B} \cup \{1\}$:

- $\gamma(\beta)$: the fraction of *inconsistent* challenge nodes among all possible nodes that could have been included into $\mathsf{chal}(\beta)$. A node $v \in [N]_0$ is inconsistent if its corresponding proof π_v is invalid.
- $\delta(\beta)$: the faction of inconsistent nodes in $\mathsf{chal}(\beta)$. We will be interested in analyzing how close $\delta(\beta)$ is to $\gamma(\beta)$.
- For $\alpha \in [0, 1)$, define
$$\eta(\beta) := \frac{i}{i_s} \cdot (1 - \alpha) \ , \tag{5}$$
where i_s is the sampling index of the last re-sampling point $\beta_s = (w_s, i_s)$. By construction, we always have $i_s \leq \log(N)$.
- Event bad_β is defined whenever a re-sampling takes place, i.e., when $\tau_r(\beta, \cdot)$ is called as in Item 1(c)iiC of the main construction,
$$\mathsf{bad}_\beta := 1 \ \Leftrightarrow \ \delta(\beta) \leq \gamma(\beta) - \eta(\beta) \ . \tag{6}$$
- $\mathsf{neighbor}(\beta) := \beta'$: set $\beta' = 1$ if β is the first element in $\mathcal{B}$, otherwise β' is defined to be the previous element in $\mathcal{B}$.

$\mathsf{Exp}^{\tau(\cdot)}_{\tilde{\mathsf{P}},\beta}(1^\lambda, 1^N) \in \{0, 1\}$*:* This is identical to its neighboring $\mathsf{Exp}^{\tau(\cdot)}_{\tilde{\mathsf{P}},\beta'}(1^\lambda, 1^N)$ except it outputs 0 if $\mathsf{bad}_\beta = 1$.

Proposition 1. *Let q upper-bounds the total number of queries $\tilde{\mathsf{P}}$ makes to $\tau_\ell(\cdot)$ in* (2)*, then*

$$\left|\Pr\left[\mathsf{Exp}^{\tau(\cdot)}_{\tilde{\mathsf{P}},0}(1^\lambda, 1^N) = 1\right] - \Pr\left[\mathsf{Exp}^{\tau(\cdot)}_{\tilde{\mathsf{P}},1}(1^\lambda, 1^N) = 1\right]\right| \leq \frac{1}{2^\lambda} + \frac{q(q-1)}{2^{\lambda+1}} \ . \tag{7}$$

Proposition 2. *For every $\beta \in \mathcal{B}$ and $\beta' = \mathsf{neighbor}(\beta)$, the following holds*

$$\left|\Pr\left[\mathsf{Exp}^{\tau(\cdot)}_{\tilde{\mathsf{P}},\beta}(1^\lambda, 1^N) = 1\right] - \Pr\left[\mathsf{Exp}^{\tau(\cdot)}_{\tilde{\mathsf{P}},\beta'}(1^\lambda, 1^N) = 1\right]\right| \leq e^{-2t \cdot \left(\frac{1-\alpha}{i_s}\right)^2} \ . \tag{8}$$

where i_s is as defined in (5)*.*

Observe that, by construction, for the first $\beta \in \mathcal{B}$, it holds that $\mathsf{Exp}^{\tau(\cdot)}_{\tilde{\mathsf{P}},\beta'} = \mathsf{Exp}^{\tau(\cdot)}_{\tilde{\mathsf{P}},1}$, and the last experiment corresponds to $\beta_s = (w_s, i_s) \in \mathcal{B}$. As the total number of such experiments is at most q, and the distance between each neighboring experiments is bounded by Proposition 2, it then holds by a simple union bound that

$$\left|\Pr\left[\mathsf{Exp}^{\tau(\cdot)}_{\tilde{\mathsf{P}},1}(1^\lambda, 1^n) = 1\right] - \Pr\left[\mathsf{Exp}^{\tau(\cdot)}_{\tilde{\mathsf{P}},\beta_s}(1^\lambda, 1^n) = 1\right]\right| \leq q \cdot e^{-2t\cdot\left(\frac{1-\alpha}{i_s}\right)^2} \quad . \tag{9}$$

For the final game with β_s, if $\mathsf{bad}_{\beta_s} = 0$, then the soundness analysis is similar to the analysis of the underlying standalone PoSW. More concretely, by definition it follows that $\eta(\beta_s) = (1-\alpha)$. Now if $\tilde{\mathsf{P}}$ made a sequence of queries of length at most $\alpha \cdot N$, then by Lemma 5, except with probability $1/2^\lambda + q^2/2^\lambda$, this must mean that $\gamma(\beta_s) \geq (1-\alpha)$, which then implies that $\delta(\beta_s) > 0$, which implies, that V rejects the proof, as the proof will contain at least one inconsistent node. Therefore,

$$\Pr\left[\mathsf{Exp}^{\tau(\cdot)}_{\tilde{\mathsf{P}},\beta_s}(1^\lambda, 1^n) = 1\right] \leq \frac{1}{2^\lambda} + \frac{q^2}{2^\lambda} \quad . \tag{10}$$

We conclude that the probability ϵ in Theorem 3 can be upper bounded by summing the probability in (7), (9), (10) and observing that $i_s \leq \log(N)$. This concludes the proof of Theorem 3.

The proofs of Propositions 1 and 2 are given Appendix B.

Acknowledgments. This work is funded by the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation program under project PICOCRYPT (grant agreement No. 101001283) and from the Spanish Government under projects PRODIGY (TED2021-132464B-I00) and ESPADA (PID2022-142290OB-I00). The last two projects are co-funded by European Union EIE, and NextGenerationEU/PRTR funds.

References

1. Abusalah, H.: Snacks for proof-of-space blockchains. In: Baldimtsi, F., Cachin, C. (eds.) Financial Cryptography and Data Security - 27th International Conference, FC 2023, Bol, Brač, Croatia, May 1-5, 2023, Revised Selected Papers, Part II. Lecture Notes in Computer Science, vol. 13951, pp. 3–17. Springer (2023). https://doi.org/10.1007/978-3-031-47751-5_1, https://doi.org/10.1007/978-3-031-47751-5_1
2. Abusalah, H., Cini, V.: An incremental posw for general weight distributions. In: Hazay, C., Stam, M. (eds.) Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part II. Lecture Notes in Computer Science, vol. 14005, pp. 282–311. Springer (2023). https://doi.org/10.1007/978-3-031-30617-4_10, https://doi.org/10.1007/978-3-031-30617-4_10

3. Abusalah, H., Fuchsbauer, G., Gazi, P., Klein, K.: Snacks: Leveraging proofs of sequential work for blockchain light clients. In: Agrawal, S., Lin, D. (eds.) Advances in Cryptology - ASIACRYPT 2022 - 28th International Conference on the Theory and Application of Cryptology and Information Security, Taipei, Taiwan, December 5-9, 2022, Proceedings, Part I. Lecture Notes in Computer Science, vol. 13791, pp. 806–836. Springer (2022). https://doi.org/10.1007/978-3-031-22963-3_27, https://doi.org/10.1007/978-3-031-22963-3_27
4. Abusalah, H., Kamath, C., Klein, K., Pietrzak, K., Walter, M.: Reversible proofs of sequential work. In: Ishai, Y., Rijmen, V. (eds.) EUROCRYPT 2019, Part II. LNCS, vol. 11477, pp. 277–291. Springer, Heidelberg (May 2019). https://doi.org/10.1007/978-3-030-17656-3_10
5. Alwen, J., Blocki, J., Pietrzak, K.: Depth-robust graphs and their cumulative memory complexity. In: Coron, J.S., Nielsen, J.B. (eds.) EUROCRYPT 2017, Part III. LNCS, vol. 10212, pp. 3–32. Springer, Heidelberg (Apr / May 2017). https://doi.org/10.1007/978-3-319-56617-7_1
6. Alwen, J., Serbinenko, V.: High parallel complexity graphs and memory-hard functions. In: Servedio, R.A., Rubinfeld, R. (eds.) 47th ACM STOC. pp. 595–603. ACM Press (Jun 2015). https://doi.org/10.1145/2746539.2746622
7. Boneh, D., Bonneau, J., Bünz, B., Fisch, B.: Verifiable delay functions. In: Shacham, H., Boldyreva, A. (eds.) CRYPTO 2018, Part I. LNCS, vol. 10991, pp. 757–788. Springer, Heidelberg (Aug 2018). https://doi.org/10.1007/978-3-319-96884-1_25
8. Boneh, D., Bünz, B., Fisch, B.: A survey of two verifiable delay functions using proof of exponentiation. IACR Commun. Cryptol. **1**(1), 7 (2024). https://doi.org/10.62056/AV7TUDHDJ, https://doi.org/10.62056/av7tudhdj
9. Cohen, B., Pietrzak, K.: Simple proofs of sequential work. In: Nielsen, J.B., Rijmen, V. (eds.) EUROCRYPT 2018, Part II. LNCS, vol. 10821, pp. 451–467. Springer, Heidelberg (Apr / May 2018). https://doi.org/10.1007/978-3-319-78375-8_15
10. Döttling, N., Garg, S., Malavolta, G., Vasudevan, P.N.: Tight verifiable delay functions. In: Galdi, C., Kolesnikov, V. (eds.) SCN 20. LNCS, vol. 12238, pp. 65–84. Springer, Heidelberg (Sep 2020). https://doi.org/10.1007/978-3-030-57990-6_4
11. Döttling, N., Lai, R.W.F., Malavolta, G.: Incremental proofs of sequential work. In: Ishai, Y., Rijmen, V. (eds.) EUROCRYPT 2019, Part II. LNCS, vol. 11477, pp. 292–323. Springer, Heidelberg (May 2019). https://doi.org/10.1007/978-3-030-17656-3_11
12. Dwork, C., Naor, M., Wee, H.: Pebbling and proofs of work. In: Shoup, V. (ed.) CRYPTO 2005. LNCS, vol. 3621, pp. 37–54. Springer, Heidelberg (Aug 2005). https://doi.org/10.1007/11535218_3
13. Erdoes, P., Graham, R.L., Szemeredi, E.: On sparse graphs with dense long paths. Tech. rep., Stanford, CA, USA (1975)
14. Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: Odlyzko, A.M. (ed.) CRYPTO'86. LNCS, vol. 263, pp. 186–194. Springer, Heidelberg (Aug 1987). https://doi.org/10.1007/3-540-47721-7_12
15. Mahmoody, M., Moran, T., Vadhan, S.P.: Publicly verifiable proofs of sequential work. In: Kleinberg, R.D. (ed.) ITCS 2013. pp. 373–388. ACM (Jan 2013). https://doi.org/10.1145/2422436.2422479
16. Pietrzak, K.: Simple verifiable delay functions. In: Blum, A. (ed.) ITCS 2019. vol. 124, pp. 60:1–60:15. LIPIcs (Jan 2019). https://doi.org/10.4230/LIPIcs.ITCS.2019.60

17. Wesolowski, B.: Efficient verifiable delay functions. In: Ishai, Y., Rijmen, V. (eds.) EUROCRYPT 2019, Part III. LNCS, vol. 11478, pp. 379–407. Springer, Heidelberg (May 2019). https://doi.org/10.1007/978-3-030-17659-4_13

A Relating Time Complexity of iPoSWs and VDFs.

Unlike VDFs, iPoSWs have a stringent efficiency requirement on the time complexity of honest parties. In an iPoSW, the prover P is not even allowed to have an additive overhead in its running time, that is, P running in time $N+c$ for any positive integer c, doesn't satisfy the definition. This very strong requirement is met by all known iPoSWs [11, 2]. However, in principle a time complexity of $N+O(\log(N))$ would still make sense. In fact, in the closely related primitive of verifiable delay functions (VDFs) [7, 16, 17], the evaluation algorithm of a VDF $f(\cdot)$ w.r.t. a delay parameter N, is allowed to output an evaluation/proof pair $(f(x), \pi)$ where π is a proof of correctness for $f(x)$ in time $N + o(N)$, i.e., the evaluator is allowed an additive overhead $o(N)$. This overhead is not a symptom of the VDF definition. In fact, the VDF evaluator of Pietrzak [16] runs in time $N + \sqrt{N}$ (and space and parallelism $\sqrt{N}$) and that of Wesolowski [17] runs in time $N + N/\log(N)$ (and space and parallelism $\log(N)$). (More general space-time trade-offs for the VDFs of Pietrzak and Wesolowski are studied in [8].) Recognizing the importance of efficiency in VDFs, [10] defines the notion of *tightness* for VDFs and gives a generic transformation that takes any *iterated* VDF whose evaluation/proof pair $(f(x), \pi)$ can be computed in $O(N)$ time and transforms it into a tight VDF: a VDF in which $(f(x), \pi)$ can be computed in time $N + O(1)$. The transform comes at the price of a $\log(N)$ multiplicative factor in the proof size as well as the space and parallel complexities of the original VDF.

Similar to tight VDFs [10], the efficiency of P in the iPoSW definition could also have been relaxed to allow P to output π in time $N + O(1)$. However, as all known iPoSWs [11, 2] achieve the stronger notion of efficiency, i.e., P runs in time exactly N, we don't see a reason to relax it. Furthermore, the possible efficiency relaxation (with an additive $O(1)$ overhead) has no impact on the results in this work.

B Omitted Proofs

B.1 Proof of Lemma 4

The `Skiplist` *Graph.* Let G_n be the `Skiplist` graph (Sect. 4). Then by definition of G_n, $V(G_n) = [N]_0$ for $N = 2^n$. By inspection, $G_{n+1} = (G_n, s_n, V_n, E_n)$ is dynamic with

$$s_n = |V(G_n)| - 1 \ , \quad V_n = \emptyset \ , \quad E_n = \{(\mathsf{source}(G_n), \mathsf{sink}(G_n \oplus s_n)) := (0, 2N)\}$$

By definition, the PoSW proof $\pi_n(v)$ of any $v \in [N]_0$ is

$$\pi_n(v) = L(P, \mathsf{parents}(P)) \text{ for } P := \mathsf{path}(v)$$

where $\mathsf{path}(v)$ denotes the shortest path from 0 to N passing through v. With pebbles on $(0, N)$, the entire graph G_{n+1} (on $[2N]_0$ vertices) can be pebbled

by a simple strategy: in *parallel* start pebbling vertices $\{1, 2, \ldots, N-1\}$ and $\{N+1, N+2, \ldots, 2N\}$. This takes time N and space 2 times the space pebbling complexity of G_n. The space pebbling complexity of G_n is roughly $\log(N)$, which is obtained by a recursive application of the aforementioned pebbling strategy – this is proven in Lemma 2 [2]. This shows that the `Skiplist` graph is incremental.

The `CP` *and* `CP`* *Graphs.* For the `CP`/`CP`* graphs (Sect. 4), by inspection, $G_{n+1} = (G_n, s_n, V_n, E_n)$ is dynamic where G_n is the tree-like DAG described in Fig. 1 with $s_n = |V(G_n)|$, $V_n = \{r\}$ where r is the root/sink of the newly formed tree, i.e., $r = \mathsf{sink}(G_{n+1})$. Now E_n is the edge set depicted in Fig.1 and formally defined in Def. 5 for the `CP` graph and in Def. 6 for the `CP`* graph. Let $H_n = G_n \oplus s_n$. What is important to highlight about E_n is that for any $(u, v) \in E_n$, it holds that $u \in \{\mathsf{sink}(G_n), \mathsf{sink}(H_n)\}$ and $v \in V(H_n) \cup V_n$. Of particular interest are the edges $(\mathsf{sink}(G_n), r), (\mathsf{sink}(H_n), r) \in E_n$ and it holds that $\mathsf{parents}_{G_{n+1}}(r) = \{\mathsf{sink}(G_n), \mathsf{sink}(H_n)\}$.

By definition of PoSW proofs based on the `CP` and `CP`* graphs (see [9, 3]), for any node v, $\pi_n(v)$ w.r.t. G_n is the "Merkle-tree opening" of v, i.e.,

$$\pi_n(v) = L(P, \mathsf{parents}(P)) \text{ for } P = \mathsf{path}(v)$$

where path denotes the shortest path from v to $\mathsf{sink}(G_n)$.

With pebbles on $\mathsf{source}(G_n)$ and $\mathsf{sink}(G_n)$, pebble in *parallel* $\{\mathsf{source}(G_n) + 1, \ldots, \mathsf{sink}(G_n)\}$ and $\{\mathsf{source}(G_n)+1, \ldots, \mathsf{sink}(G_{n+1})\}$ in time $t_1 := |V(G_{n+1})| - |V(G_n)|$ and space $O(\log(t_1))$. (The space pebbling complexity of G_n is formally proven in Lemma 3 [9])). This shows that the `CP`/`CP`* graphs are incremental.

B.2 Proof of Proposition 1

The proof boils down to either finding a collision under τ_ℓ or that $\tilde{\mathsf{P}}$ guesses the output of τ_ℓ. The latter happens with probability $1/2^\lambda$ and the former with probability $q(q-1)/2^{\lambda+1}$.

B.3 Proof of Proposition 2

To prove Proposition 2, we state and use the re-sampling lemma as formulated in [2], which is itself a restatement from [11].

Lemma 6 ([2]). *Let t be an integer, $b \in \{0, 1\}$, $\delta_b \in [0, 1]$, and sets U_b s.t. $|U_b| = t$, $U_0 \cap U_1 = \emptyset$ and U_b contains elements that are either* consistent *or* inconsistent *with δ_b being the fraction of inconsistent elements in U_b, and*

$$\delta_b > \gamma_b - \eta_b \ , \tag{11}$$

for some global values $\gamma_b, \eta_b \in [0, 1]$.

We sample from $U_0 \cup U_1$ without replacement in t draws a set U, with $|U| = t$, and let X be the random variable indicating the number *of inconsistent elements in U. Then an arbitrary $\eta \in [0, 1]$, we have*

$$\Pr\left[X \leq t \cdot \left(\frac{\gamma_0 + \gamma_1}{2} - \eta\right)\right] \leq e^{-2t\cdot\left(\eta - \frac{\eta_0+\eta_1}{2}\right)^2} \ . \tag{12}$$

Proof (Proposition 2). The proof amounts to bounding the probability of bad_β. For $\beta = (w, i) \in \mathcal{B}$, recall we re-sample t indices $S_{w,i}$ from $S_{u,i-1} \cup S_{v,i-1}$ for some $(\beta_0 := (u, i-1), \beta_1 := (v, i-1)) \in \mathcal{M}$ for which $\mathsf{bad}_{\beta_0} = \mathsf{bad}_{\beta_1} = 0$, for otherwise we would not be in this game. Then, for $b \in \{0, 1\}$, we have that $\gamma_b := \gamma(\beta_b), \delta_b := \delta(\beta_b), \eta_b := \eta(\beta_b)$ satisfy

$$\delta_b > \gamma_b - \eta_b = \gamma_b - \frac{i-1}{i_s} \cdot (1-\alpha) \ . \tag{13}$$

We now apply Lemma 6 with $U_0 := S_{u,i-1}, U_1 := S_{v,i-1}$, and $\eta := \eta(\beta) := \frac{i}{i_s} \cdot (1-\alpha)$ and noting that $\gamma := \gamma(\beta) := (\gamma_0 + \gamma_1)/2$, we get

$$\begin{aligned} \Pr[\mathsf{bad}_\beta = 1] &= \Pr[t \cdot \delta(\beta) \leq t \cdot (\gamma - \eta)] \\ &\leq e^{-2t \cdot \left(\frac{i \cdot (1-\alpha)}{i_s} - \frac{(i-1) \cdot (1-\alpha)}{i_s}\right)^2} \\ &= e^{-2t \cdot \left(\frac{1-\alpha}{i_s}\right)^2} \ . \end{aligned} \tag{14}$$

□

C Examples

In this section, we illustrate certain elements of the iPoSW transform from Sect. 7.2. In particular, we give example invocations of Algorithm 1 which determines the re-sampling nodes. We show how the incremental sampling of sets is executed by the prover algorithm P. We also illustrate the way the sampling sets are implicitly stored as indices and how these are mapped to actual challenge nodes. We also illustrate the meaning of the bookkeeping indices $\mathcal{I}$ the prover P produces. We apply these to the `Skiplist` and `CP`/`CP`* graphs (Sect. 4).

The Skiplist Graph. We first apply Algorithm 1 to determine all the re-sampling nodes. For simplicity, we consider the `Skiplist` graph from Fig. 2 on vertices $[8]_0$. Let's assume G_0 to be the graph on vertices $[2]_0$, then $s_n = |V(G_n)| - 1$ and $t = s_0 = 2$, and $V_n = \emptyset$. We conclude that

- For $v \in \{0, 1, 3, 5, 6, 7\}$, $\mathcal{M} = \emptyset$. These are not re-sampling nodes.
- For $v = 4$, $\mathcal{M} = \{((4, 0), (2, 0))\}$. This is the first re-sampling node.
- For $v = 8$, $\mathcal{M} = \{((8, 1), (4, 1)), ((8, 0), (6, 0))\}$. This is the second re-sampling node.

The prover P records the re-sampling nodes and their labels in a set $\mathcal{R} = \{(4, L(4)), (8, L(8)))\}$. The labels in $\mathcal{R}$ will be needed for the verifer V to check the veracity of the incremental sampling process.

We show how we generate challenges incrementally and how to produce a bookkeeping set of values that will aid the verifier in verifying the veracity of the incremental sampling. Whenever we re-sample a set from two sets, we ignore the sampling randomness and give an arbitrary example.

We start indexing sets from 1, i.e., the first element in a set S is $S[1]$. We also assume sets are ordered when convenient. All sampling sets $S_{v,i}$ below contain *indices of elements rather than actual elements/vertices*. For each $S_{v,i}$, we define, for the sake of illustration, its corresponding set of values $S'_{v,i}$. Primed sets contain values and non-primed sets contain indices. In the main construction, we store indices as they are cheaper to represent than actual values.

1. Sample $S_{4,1} = \{1, 3\}$ from $S'_{2,0} = \{1, 2\}$ and $S'_{4,0} = \{3, 4\}$. These two sets are implicitly defined and $S_{4,1}$ is formed by taking elements number $1, 3$ from (ordered) $S'_{2,0} \cup S'_{4,0}$. In absolute terms, $S'_{4,1} = \{1, 3\}$.
2. Sample $S_{8,1} = \{2, 4\}$ from $S'_{6,0} = \{5, 6\}$ and $S'_{8,0} = \{7, 8\}$. These two sets are implicitly defined and $S_{8,1}$ is formed by taking elements number $2, 4$ from $S'_{6,0} \cup S'_{8,0}$. In absolute terms, $S'_{8,1} = \{6, 8\}$.
3. Sample $S_{8,2} = \{2, 4\}$ is sampled from $S_{8,1} \cup S_{4,1}$. In absolute value, $S'_{8,2} = \{3, 8\}$.

That is, the final challenges are $\{3, 8\}$ and P's output $\pi := (\mathcal{L}_{8,2}, \mathcal{I}_{8,2}, \mathcal{R}, 2)$ is interpreted as follows

- $\mathcal{L}_{8,2}[1], \mathcal{L}_{8,2}[2]$ contain the PoSW proofs for challenges $3, 8$, respectively.
- $\mathcal{I}_{8,2}[i]$ contains the sampling indices of the ith challenge. These are initially defined by (3) and then appended by Algorithm 2. We illustrate these here:
  - $\mathcal{I}_{8,2}[1] = (1, 2, 1)$ corresponding to challenge 3. This is interpreted as follows. Challenge 3 is the first element in the first set $S'_{4,0}$ it belonged to; the second element in the second set $S'_{4,1}$ it belonged to; the first element in the third set $S'_{8,2}$ it belonged to.
  - $\mathcal{I}_{8,2}[2] = (2, 2, 2)$ corresponding to challenge 8.

Relying on Algorithm 3, the verifier V uses these indices $\mathcal{I}_{8,2}$ alongside the sampling nodes $\mathcal{R}$ to verify that the set of challenges $\{3, 8\}$ were computed correctly, and then accepts if and only if the corresponding proofs $\mathcal{L}_{8,2}$ are valid.

The $\mathtt{CP}/\mathtt{CP}^*$ Graph. Let Π' be the PoSW over the $\mathtt{CP}$ graph. Everything in this section also applies as is to Π' over the $\mathtt{CP}^*$ graph.

We first apply Algorithm 1 to determine all the re-sampling points. Let $\mathcal{G} = (G_n)_{n \in \mathbb{N}}$ be the $\mathtt{CP}$ graph family and let G_0 be defined such that $V(G_0) = [2]_0$. We note that $s_n = |V(G_n)|$, $t = |V(G_0)| = 3$, $V_n = \{2 \cdot |V(G_{n-1})|\}$. ($E_n$ would not matter for this discussion.). We conclude that

- For $v \in \{0, 1, 2, 3, 4, 5, 7, 8, 9, 10, 11, 12\}$, $\mathcal{M} = \emptyset$. Such v's are not re-sampling nodes.
- For $v = 6$, $\mathcal{M} = \{((5, 0), (2, 0))\}$. This is the first re-sampling node.
- For $v = 13$, $\mathcal{M} = \{((12, 0), (9, 0))\}$. This is the second re-sampling node.
- For $v = 14$, $\mathcal{M} = \{((13, 1), (6, 1))\}$. This is the third re-sampling node.

The prover P records the re-sampling nodes and their labels in a set $\mathcal{R} = \{(6, L(6)), (13, L(13)), (14, L(14))\}$. The labels in $\mathcal{R}$ will be needed for the verifer V to check the veracity of the incremental sampling process.

We show how we generate challenges incrementally and how to produce a bookkeeping set of values that will aid the verifier in verifying the veracity of the incremental sampling. Whenever we re-sample a set from two sets, we ignore the sampling randomness and give an arbitrary example.

We start indexing sets from 1, i.e., the first element in a set S is $S[1]$. We also assume sets are ordered when convenient. All sampling sets $S_{v,i}$ below contain *indices of elements rather than actual elements/vertices*. For each $S_{v,i}$, we define, for the sake of illustration, its corresponding set of values $S'_{v,i}$. Primed sets contain values and non-primed sets contain indices. In the main construction, we store indices as they are cheaper to represent than actual values.

1. Sample $S_{6,1} = \{1, 2, 4\}$ from $S'_{2,0} = \{0, 1, 2\}$ and $S'_{5,0} = \{3, 4, 5\}$. These two sets are implicitly defined and $S_{6,1}$ is formed by taking elements number $1, 2, 4$ from (ordered) $S'_{2,0} \cup S'_{5,0}$. In absolute terms, $S'_{6,1} = \{0, 1, 3\}$.
2. Sample $S_{13,1} = \{2, 3, 6\}$ from $S'_{9,0} = \{7, 8, 9\}$ and $S'_{12,0} = \{10, 11, 12\}$. These two sets are implicitly defined and $S_{12,1}$ is formed by taking elements number $2, 3, 6$ from (ordered) $S'_{9,0} \cup S'_{12,0}$. In absolute terms, $S'_{13,1} = \{8, 9, 12\}$.
3. Sample $S_{14,2} = \{3, 4, 6\}$ is sampled from $S_{6,1} \cup S_{13,1}$. In absolute value, $S'_{14,2} = \{3, 8, 12\}$.

That is, the final challenges are $\{3, 8, 12\}$ and P's output $\pi := (\mathcal{L}_{14,2}, \mathcal{I}_{14,2}, \mathcal{R}, 2)$ is interpreted as follows

- $\mathcal{L}_{14,2}[1], \mathcal{L}_{14,2}[2], \mathcal{L}_{14,2}[3]$ contain the PoSW proofs for challenges $3, 8, 12$, respectively.
- $\mathcal{I}_{14,2}[i]$ contains the sampling indices of the ith challenge. These are initially defined by (3) and then appended by Algorithm 2. We illustrate these here:
  - $\mathcal{I}_{14,2}[1] = (1, 3, 1)$ corresponding to challenge 3.
  - $\mathcal{I}_{14,2}[2] = (2, 1, 2)$ corresponding to challenge 8. This is interpreted as follows. Challenge 8 is the second element in the first set $S'_{9,0}$ it belonged to; the first element in the second set $S'_{13,1}$ it belonged to; the second element in the third set $S'_{14,2}$ it belonged to.
  - $\mathcal{I}_{14,2}[1] = (3, 3, 3)$ corresponding to challenge 12.

Relying on Algorithm 3, the verifier V uses these indices $\mathcal{I}_{14,2}$ alongside the sampling nodes $\mathcal{R}$ to verify that the set of challenges $\{3, 8, 12\}$ were computed correctly, and then accepts if and only if the corresponding proofs $\mathcal{L}_{14,2}$ are valid.